GeneralizedNotationNotation: A Text Language for Active Inference Models

A text language and modular processing pipeline for Active Inference generative models

Daniel Ari Friedman

Active Inference Institute

daniel@activeinference.institute

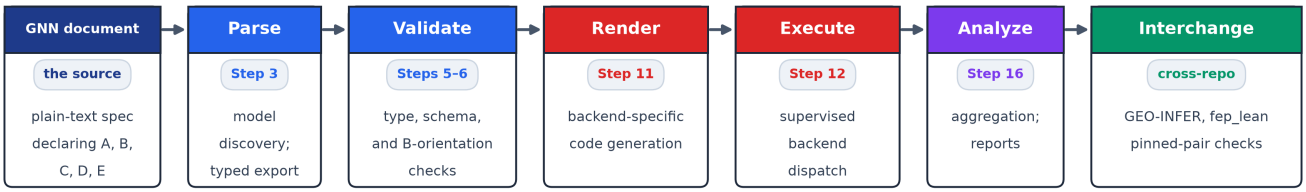
ORCID: 0000-0001-6232-9096

DOI: 10.5281/zenodo.7803313

2026-10-08

Generalized Notation Notation at a Glance

GNN v4.1.0 — one text specification, many traceable realizations



New in validation — the B-tensor orientation check reads every B literal as column-stochastic B[s′, s, a] (the pymdp convention); --transpose-b maps row-stochastic textbook literals onto the canonical order instead of failing them. New in execution — the bnlearn lane generates runnable Python from the same parsed model and skips with a recorded status when the bnlearn runtime is absent.

25 pipeline steps (0–24) · 9 model families · 12 registered render backends, 12 render-target adapters at Step 12 · 164 MCP tools

Contents

1 Abstract **2**

2 Introduction **3**
- 2.1 Motivation . . . 3
- 2.2 The GNN Approach . . . 3
- 2.3 Contributions . . . 3
- 2.4 Reader Orientation . . . 4

3 System Context **5**
- 3.1 The GNN Language . . . 5
- 3.2 Model Kinds . . . 5
  - 3.2.1 The Discrete Categorical Kind . . . 6
  - 3.2.2 The Continuous Linear-Gaussian Kind . . . 7
  - 3.2.3 Multi-Agent, Hierarchical, and Structural Kinds . . . 7
  - 3.2.4 Reading Exemplars by Kind . . . 8
  - 3.2.5 What the Kind Classification Governs . . . 8
- 3.3 The Processing Pipeline . . . 9
- 3.4 The Triple Play . . . 11

4 Methods **12**
- 4.1 Parsing and Multi-Format Export . . . 12
- 4.2 Type Checking and Validation . . . 12
- 4.3 Rendering to Multiple Backends . . . 12
  - 4.3.1 Rendering by Model Kind . . . 13
- 4.4 Execution . . . 15
  - 4.4.1 Executing by Model Kind . . . 15
- 4.5 Long-Running Orchestration Contracts . . . 15
- 4.6 Analysis, Visualization, and Reporting . . . 16
- 4.7 Reproducibility and Auto-Injection . . . 16

5 Artifacts and Evidence **18**
- 5.1 Model-Family Coverage . . . 18
- 5.2 Semantic-Fidelity and Cross-Framework Reliability Gates . . . 19
- 5.3 Repository Scale . . . 20
- 5.4 Claim Discipline . . . 21

6 Reproducibility **22**
- 6.1 Pipeline Smoke Run . . . 22
- 6.2 Validation Gates . . . 22
- 6.3 Manuscript Reproducibility . . . 23
- 6.4 Reproducibility Contract . . . 23

7 Limitations and Next Steps **24**
- 7.1 Current Limitations . . . 24
- 7.2 Next Steps . . . 24

8 Supplemental Source Surface **26**
- 8.1 Authored Source Versus Generated Output . . . 26

9 Symbols and Glossary **27**
- 9.1 Language Constructs . . . 27
- 9.2 Symbols by Model Kind . . . 28
- 9.3 Ontology Bindings and Implementations . . . 29

10 References **30**

1 Abstract

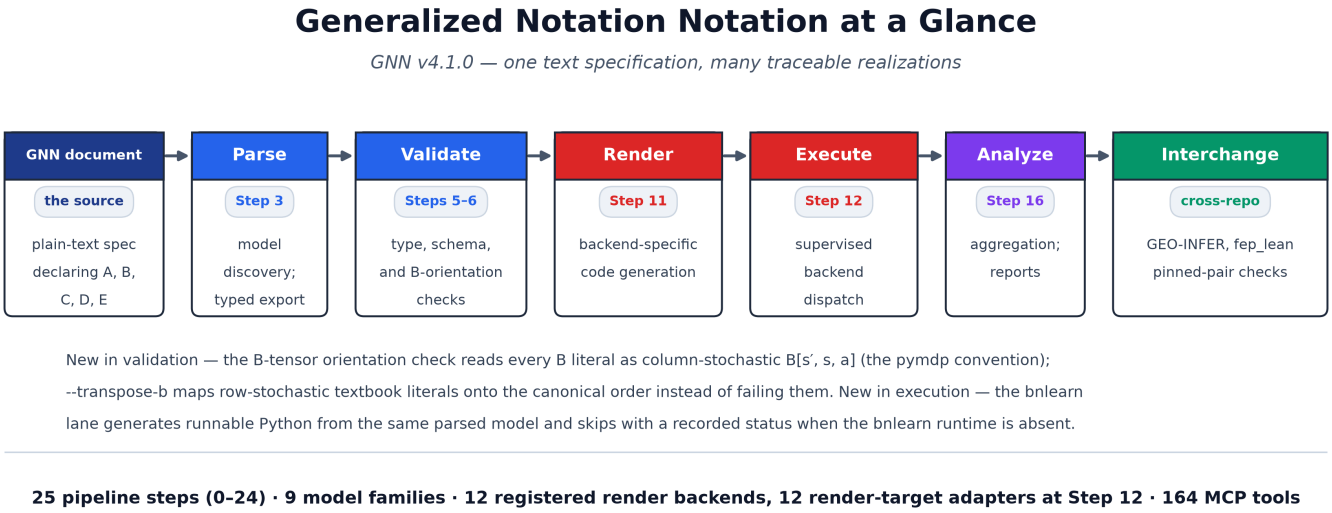


Figure 1: The GeneralizedNotationNotation (GNN) pipeline at a glance, generated from the producer's token map at commit d20b76a54. A plain-text specification — the `StateSpaceBlock` declaring the model's tensors, `A`, `B`, `C`, `D`, and `E` for the discrete categorical kind of eq. 1 through eq. 8 and its own matrix vocabulary for the other model kinds — is parsed (Step 3), type-checked and validated (Steps 5–6), rendered as backend-specific code (Step 11), executed (Step 12), and analyzed (Step 16), before artifacts flow into the cross-repository interchange checks against the pinned GEO-INFER and fep_lean pairs. Read the cards left to right: each header names the stage, each pill names the pipeline step (or the source role), and the body states what the stage produces. The validation stage includes the B-tensor orientation check — every discrete `B` literal is read as column-stochastic `B[s', s, a]`, and `--transpose-b` maps row-stochastic textbook literals onto that canonical order — and the execution stage includes the bnlearn lane, which generates runnable Python from the same parsed model and skips with a recorded status when its runtime is absent. The footer states the scale the panel summarizes: 25 pipeline steps, 9 model families, 12 registered render backends with 12 render targets having Step 12 execution adapters, and 164 Model Context Protocol tools.

Active Inference offers a unifying account of perception, learning, and action under the free energy principle [Friston, 2010], yet the generative models at its core are still communicated ad hoc: scattered across prose descriptions, bespoke notebooks, and framework-specific code that rarely agree. This fragmentation makes published models hard to reproduce, compare, or port between tools, and it raises the barrier for newcomers learning the formalism [Parr et al., 2022]. GeneralizedNotationNotation (GNN) addresses this gap with a standardized, human- and machine-readable text language for specifying Active Inference generative models, paired with a 25-step processing pipeline that transforms a single specification into validation, visualization, simulation, and analysis artifacts [Smékal and Friedman, 2023]. A GNN file denotes a generative model, and the language lets that model take the several shapes the field actually uses: the notation blocks a file declares — categorical `A/B/C/D[/E]` tensors, linear-Gaussian system matrices `F/H/Q/R` with a Gaussian prior, per-agent and per-level compositions — fix its model kind, which the pipeline classifies structurally and carries through rendering and execution, recording explicitly which registered backends can render and run a model of each kind and which report it unsupported. GNN's "Triple Play" treats each model as three coordinated views — a textual specification, graphical visualizations, and executable cognitive models — so that outputs share an authored source and can be checked for consistency through identity, parameter, semantic, and numerical receipts. The framework spans 38 exemplar specifications (29 discrete-state and 9 continuous linear-Gaussian) across 9 model families and 12 registered rendering backends, with explicit gates for cross-format semantic fidelity and cross-framework reliability. GNN 3.0.0 layered safe-by-design long-running orchestration on top of this language and pipeline — durable observation streams, resumable run sessions, and auditable container plans that generate, validate, and replay data only, with no live infrastructure mutation. We describe the language, the model kinds it denotes, the pipeline architecture, and the validation contracts and source-bound checks used to assess specification round trips and admitted executions across simulation backends — with coverage gaps recorded as explicit profiled-unsupported statuses rather than silently omitted — providing inspectable infrastructure for reproducing and comparing Active Inference models within the recorded capability and verification boundaries.

2 Introduction

2.1 Motivation

Active Inference has matured into a broad research program spanning the free energy principle [Friston, 2010], discrete state-space formulations of perception and action [Da Costa et al., 2020], and a growing body of pedagogy and reference implementations [Parr et al., 2022, Smith et al., 2022]. Recent graphical-specification and message-passing work makes the same practical pressure visible: active-inference agents need representations that can be inspected as model structure and then carried into executable inference updates without being reauthored by hand [Koudahl et al., 2023, van de Laar et al., 2023, Bagaev and de Vries, 2021]. As the field has grown, so has a quieter problem: the models themselves are difficult to share, reproduce, and compare. A generative model that lives only as a tangle of matrix definitions inside one author's script, a diagram in a slide deck, and a paragraph of prose in a paper has no single authoritative form. The same model is described three times, in three incompatible media, with no guarantee that they agree. When a reader wants to re-run that model in a different toolkit, they must reverse-engineer it from whichever fragment they happen to have.

This is a reproducibility and interoperability crisis specific to the structure of Active Inference work. The discipline depends on precisely specified state spaces, observation and transition tensors, prior preferences, and policy structures; small notational ambiguities propagate into materially different behaviour. The heterogeneity of the models themselves compounds the problem: the field works in several model kinds at once — discrete categorical POMDPs over finite state spaces, continuous linear-Gaussian state-space models, multi-agent systems whose agents each carry their own generative vocabulary, hierarchical compositions layered over timescales — and each kind attracts its own notation, its own notebooks, and its own simulation stack. Yet the community has lacked a notation that is simultaneously human-readable, machine-parseable, and faithful to the underlying mathematics across those kinds. Generalized Notation Notation (GNN) was introduced to fill exactly that gap [Smékal and Friedman, 2023]: a standard, text-based way to write down an Active Inference generative model once, such that every downstream use derives from the same source of truth.

2.2 The GNN Approach

GNN treats the model specification as a first-class artifact. A model is written in a plain-text language whose syntax captures the components an Active Inference modeller actually reasons about — state factors, observation modalities, the matrices that link them, control structure, and the temporal organization of the model — and whose declared matrix vocabulary fixes which kind of generative model the file denotes: categorical `A/B/C/D[/E]` tensors for a discrete state-space model, linear-Gaussian system matrices for a continuous one, per-agent and per-level compositions for structured variants. Because the specification is text, it lives comfortably in version control, diffs cleanly across revisions, and can be authored and reviewed by humans without specialized tooling.

The defining commitment of GNN is what the project calls the Triple Play: a single text specification is the common origin for three coordinated renderings of the same model. The text form is the authoritative, editable description. From it, GNN produces graphical visualizations that expose the model's factor and dependency structure for inspection and communication. And from the same source it produces executable cognitive models that can actually be run, so that the diagram, the equations, and the running code are all generated from one parsed document rather than three artifacts that merely claim to describe the same model. The Triple Play turns a model from a description scattered across media into one specification with traceable projections (see fig. 3). Shared source identity supports their comparison; parameter, orientation, model-kind, and numerical checks are still required to establish fidelity.

2.3 Contributions

This work contributes a standard notation together with the infrastructure that makes it usable in practice:

- **A parseable, human-readable notation** for Active Inference generative models, whose text form is authoritative and from which every other representation is derived.
- **A model-kind taxonomy with per-kind rendering and execution semantics** — discrete categorical, continuous linear-Gaussian, multi-agent, hierarchical, learning, and structural kinds, classified structurally from the declared notation and rendered or recorded-unsupported per backend — so that what a specification can do on a given framework is a checkable property of the model's kind, not a trial-and-error discovery.
- **A 25-step processing pipeline** (0–24), whose stages are implemented across 44 source packages, that carries a specification from parsing and validation through visualization, rendering, and execution.
- **12 rendering backends** (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn) that materialize a single GNN specification as backend-specific model code, of which 12 (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn) have Step 12 execution adapters. Actual execution requires an admitted model, a ready runtime, and a source-bound run receipt.
- **164 Model Context Protocol tools** that expose the pipeline's capabilities to agentic and programmatic clients, so the notation and its tooling are directly accessible to automated workflows.

- **9-family reliability gates** that exercise the pipeline against a curated set of model families (basics, discrete, continuous, hierarchical, multiagent, precision, structured, gridworld, scaling-study), turning interoperability claims into checks that must pass rather than assertions that are merely made.

2.4 Reader Orientation

The remainder of this manuscript is organized to move from language to mechanism to evidence. sec. 3 states the notation and the model kinds it denotes — the discrete categorical factorization, its four matrices, and its two free-energy objectives as explicit equations (eq. 1 through eq. 8), the continuous linear-Gaussian state-space kind with its optional closed-loop control structure (eq. 9 through eq. 11), and the multi-agent, hierarchical, learning, and structural compositions — and it presents the model-kind table (tbl. 1), the pipeline structure (fig. 2), the Triple Play (fig. 3), and the per-step responsibility table (tbl. 2). sec. 4 follows a specification through the pipeline — parsing, kind-scoped type checking, rendering, execution, and reporting — and presents the backend registry (tbl. 3), the per-kind framework capability matrix (tbl. 4), the backend registry surface (fig. 4), and the long-running orchestration contracts (fig. 5). sec. 5 reports what the project has produced and how each number is grounded, with the model-family registry (tbl. 5), the family-by-framework coverage (fig. 6), and repository-scale metrics (fig. 7). fig. 1 condenses this same arc — specification, validation, rendering, execution, analysis, and interchange — into the summary panel at the head of the manuscript. Claims of independent re-execution are addressed in sec. 6, the boundaries of the current system in sec. 7, the symbol and construct tables in sec. 9, and full source details in sec. 10. Read in this order, the manuscript moves from why a standard notation is needed, through how GNN realizes it across model kinds, to the scoped evidence available for the notation and its implementations.

3 System Context

Generalized Notation Notation (GNN) couples a small, declarative text language for generative models in the Active Inference lineage to a pipeline of source-processing transformations and bounded execution that produces validation, visualization, simulation, and analysis artifacts [Smékal and Friedman, 2023]. The language gives a model one canonical written form; the pipeline gives that form many executable and graphical realizations. This section describes both halves of the architecture and the way they meet, and it states the mathematical objects the language is obliged to carry — per model kind, because the language is not tied to a single shape of generative model.

3.1 The GNN Language

A GNN model is a plain-text document organized into named sections that together pin down the generative model its author intends. The `StateSpaceBlock` declares the variables of the model and their dimensions — hidden states, observations, control factors, policies, or the system matrices of a continuous model — establishing the shape of every tensor that follows. The `Connections` section records the directed and undirected dependencies among those variables, the edges of the underlying factor graph that downstream tools read to lay out diagrams and wire up inference. Which family of generative model a file declares is decided by the notation the file carries, not by its filename or its directory: the declared matrix vocabulary is what the pipeline reads to classify the model, as sec. 3.2 specifies. The full construct vocabulary is catalogued in tbl. 6.

3.2 Model Kinds

Every GNN file denotes one generative model, and the language lets that model take several distinct mathematical shapes. A file whose `StateSpaceBlock` declares the categorical tensors `A`, `B`, `C`, `D` — optionally `E` — denotes a discrete state-space generative model. A file that declares the system matrices `F`, `H`, `Q`, `R` together with a Gaussian prior over the initial state (`prior_mean`, `prior_cov`) denotes a continuous linear-Gaussian state-space model. Per-agent and per-level declarations (`A_agent1`, `A_level1`, …) compose the categorical vocabulary into multi-agent and hierarchical structures. A file that declares boundary structure only — neither a categorical nor a continuous parameterization — is a structural wrapper: it names the model's interfaces but carries no renderable form.

The pipeline classifies each parsed specification structurally, and it does so from typed fields alone: the raw `## GNNSection` value, the declared matrix-key patterns, explicit agent counts, and explicit `## ModelParameters` keys. The classifier (`detect_model_kind` in `src/gnn/render/pomdp_contract.py`) resolves kinds in a fixed precedence order — multi-agent, then hybrid, then hierarchical, then continuous, then nonstationary, then learning, then factored, then structural, then the flat single-factor case. This single-winner interface identifies the most specific kind; composition-aware dispatch consults the complete `detect_model_kinds` set to admit or refuse the whole declared composition. Two properties of this classification matter for everything downstream. First, it is structural, not textual: prose in a model name or annotation never changes how a model renders, so a passing mention of "Dirichlet" in free text cannot reroute a specification into the parameter-learning kind. Second, it is computed from the parsed specification, not asserted by the author: the same file classifies the same way on every machine, which is what lets rendering and execution behavior be stated per kind rather than per file. The taxonomy, its notation blocks, exemplar folder, and rendering and execution reach per kind are enumerated in tbl. 1. Registry flags do not establish runtime availability; execution receipts do.

Table 1: The model kinds the pipeline represents and executes, one row per kind. Notation blocks are the parameterization each kind declares; exemplar folders are the committed corpus under `input/gnn_files/`. Renderer(s)/Executor(s) cells are generated from the registry flags (see tbl. 4): the two parameterization rows carry their own flag sets, excluding continuous-only targets from categorical coverage. These flags do not establish universal multi-agent support: THRML accepts independent categorical components, and coupled models require a backend-specific representation and acceptance evidence. Native independent linear-Gaussian agent exemplars are verified on JAX and RxInfer in the v4 acceptance ledger; `recursive/` is reserved for bounded `--autonomous` proposal-loop runs and holds no committed models.

Model kind	Notation block	Exemplar folder	Renderer(s)	Executor(s)
Discrete categorical	`A`/`B`/`C`/`D`[/`E`] (column-stochastic `B` slices)	`input/gnn_files/discrete/`	PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, THRML	PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, THRML
Continuous linear-Gaussian	`F`/`H`/`Q`/`R` + priors (optional closed-loop pair)	`input/gnn_files/continuous/`	RxInfer.jl, JAX, PyTorch, NumPyro, Stan, cpomdp, ngc-learn	RxInfer.jl, JAX, PyTorch, NumPyro, Stan, cpomdp, ngc-learn

Model kind	Notation block	Exemplar folder	Renderer(s)	Executor(s)
Multi-agent	`nr_agents` + per-agent categorical or linear-Gaussian parameters	`input/gnn_files/multiagent/`	Model/backend dependent composition	Model/backend dependent; explicit acceptance required
Recursive	—	`input/gnn_files/recursive/`	—	—

3.2.1 The Discrete Categorical Kind

A discrete specification denotes a partially observable Markov decision process over a horizon T, factorized as in eq. 1 following the standard formulation [Da Costa et al., 2020, Smith et al., 2022].

$$P(o_{1:T},\, s_{1:T},\, \pi) \;=\; P(\pi)\, P(s_1) \prod_{t=1}^{T} P(o_t \mid s_t) \prod_{t=2}^{T} P(s_t \mid s_{t-1},\, \pi) \tag{1}$$

The four matrices named in the `StateSpaceBlock` are exactly the factors of eq. 1, and this correspondence is what makes the notation translatable rather than merely descriptive. The `A` matrix is the likelihood, a column-stochastic map from hidden states to observation outcomes, given in eq. 2.

$$P(o_t = i \mid s_t = j) \;=\; A_{ij}, \qquad \sum_i A_{ij} = 1 \tag{2}$$

The `B` matrix is the controlled transition dynamics: one column-stochastic slice per action, as in eq. 3. The canonical reading of the tensor is column-stochastic, `B[next_state][previous_state][action]`, matching the convention of the discrete message-passing libraries the pipeline targets [Heins et al., 2022]; validation records every declared tensor's detected orientation and can transpose row-stochastic textbook literals on request, as described in sec. 4.

$$P(s_{t+1} = i \mid s_t = j,\, u_t = k) \;=\; B^{(k)}_{ij}, \qquad \sum_i B^{(k)}_{ij} = 1 \tag{3}$$

The `C` vector holds log-preferences over observations, which enter inference as the biased outcome distribution of eq. 4; the `D` vector is the prior over initial hidden states, eq. 5.

$$\tilde{P}(o_t = i) \;=\; \sigma(C)_i \;=\; \frac{\exp C_i}{\sum_j \exp C_j} \tag{4}$$

$$P(s_1 = i) \;=\; D_i, \qquad \sum_i D_i = 1 \tag{5}$$

Given those factors, perception is approximate Bayesian inference: a variational posterior $Q(s)$ is fitted by minimizing the variational free energy of eq. 6, which decomposes into a complexity term and an accuracy term [Friston, 2010].

$$F[Q] \;=\; \underbrace{D_{\mathrm{KL}}[\,Q(s) \,\|\, P(s)\,]}_{\text{complexity}} \;-\; \underbrace{\mathbb{E}_{Q(s)}[\ln P(o \mid s)]}_{\text{accuracy}} \tag{6}$$

Action is expected-free-energy minimization: each policy π is scored over future steps τ by eq. 7, whose two terms are the information gain a policy is expected to yield and the extent to which its predicted outcomes match `C` [Da Costa et al., 2020].

$$G(\pi) \;=\; \underbrace{-\,\mathbb{E}_{Q(o_\tau, s_\tau \mid \pi)}[\ln Q(s_\tau \mid o_\tau, \pi) - \ln Q(s_\tau \mid \pi)]}_{\text{epistemic value}} \;-\; \underbrace{\mathbb{E}_{Q(o_\tau \mid \pi)}\left[\ln \tilde{P}(o_\tau)\right]}_{\text{pragmatic value}} \tag{7}$$

The policy posterior then combines those scores with the habit prior `E` declared in the specification, as in eq. 8, and the selected action u is sampled from it.

$$Q(\pi) \;=\; \sigma(\ln E - G) \tag{8}$$

The discrete categorical kind is the language's original and most densely exercised target: 29 of the 38 exemplar specifications in the corpus are discrete-state. The discrete family folder holds the kind's core set — from minimal `markov_chain.md` and

hmm_baseline.md fixtures through epistemic planning (tmaze_epistemic.md) and deep planning horizons (deep_planning_horizon.md) to the canonical full agent (actinf_pomdp_agent.md) — and further categorical fixtures populate the basics, precision, structured, gridworld, and scaling-study corpora. The kind composes upward as well: multiple independent state factors (a factored model), per-level and per-agent matrix declarations (sec. 3.2.3), and Dirichlet priors that turn a declared matrix into a learned latent (sec. 7) are all extensions of this same categorical substrate.

Recent work continues to refine how expected-free-energy objectives relate to variational inference and to alternative but equivalent formulations, which is why GNN keeps the objects of eq. 1 through eq. 8 explicit in the notation rather than burying them in backend-specific code [Champion et al., 2024, Nuijten et al., 2026a,b]. A `ModelParameters` section fixes the scalars those objects depend on — factor cardinalities and precision terms — while a `Time` section declares whether the model is static or dynamic and, if dynamic, how the horizon T and its discretization are organized. Because every one of these sections is explicit text, a GNN file is at once human-readable, diffable under version control, and unambiguous to a parser — the property that makes the declared source contract inspectable; numerical replay also depends on the chosen inference and runtime configuration.

3.2.2 The Continuous Linear-Gaussian Kind

A file whose `StateSpaceBlock` declares `F`, `H`, `Q`, `R` together with `prior_mean` and `prior_cov` denotes a continuous-state linear-Gaussian state-space model stepped in discrete time: a Gaussian latent state $x_t \in \mathbb{R}^n$ evolves linearly, and a Gaussian observation $y_t \in \mathbb{R}^m$ reads it out linearly.

$$x_1 \sim \mathcal{N}(\mu_0, \Sigma_0), \qquad x_t = F\,x_{t-1} + u_{t-1} + w_t, \qquad w_t \sim \mathcal{N}(0, Q) \tag{9}$$

$$y_t = H\,x_t + v_t, \qquad v_t \sim \mathcal{N}(0, R) \tag{10}$$

Here `F` is the state-transition matrix, `H` the observation matrix, `Q` the process-noise covariance, and `R` the observation-noise covariance; `prior_mean` (μ_0) and `prior_cov` (Σ_0) fix the Gaussian prior over the initial state, so the model's randomness is declared entirely in its parameterization rather than in categorical tables. The shape contract mirrors the semantics: `prior_mean` must carry exactly n entries, and `Q`, `R`, and `prior_cov` must be $n \times n$, $m \times m$, and $n \times n$ respectively — a mismatch is a validation-time diagnostic in the notation, not a runtime surprise inside a numerical backend.

The kind also carries an optional closed-loop control structure. When a specification declares `goal_mean` ($\mu^\star$, the preferred state) and `control_gain` (k, a scalar proportional gain) alongside a control input u_t, the rendered model closes the loop on beliefs: the controller steers the filtered posterior mean toward the preferred state.

$$u_t \;=\; k\,(\mu^\star - \mu_t), \qquad \mu_t = \mathbb{E}[x_t \mid y_{1:t}] \tag{11}$$

This is how `input/gnn_files/continuous/continuous_navigation.md` is built: a two-dimensional navigator whose position persists between steps ($F = I$), whose observation is a noisy identity readout of that position, and whose control input pushes the believed position toward `goal_mean` with gain `control_gain`. The other continuous exemplars are passive: `predictive_coding_agent.md` and `stochastic_dynamics.md` filter without steering. In total the corpus ships 9 continuous linear-Gaussian exemplars in the continuous family folder. Symbolically the kind reuses letters the discrete vocabulary already claims — most prominently `F`, which names the state-transition matrix here and the variational free energy in eq. 6 — and sec. 9 resolves that collision by scoping each symbol table to its kind.

3.2.3 Multi-Agent, Hierarchical, and Structural Kinds

The remaining kinds are structural compositions of the two parameterization vocabularies above, and the classifier recognizes each by its declared key pattern rather than by convention.

Multi-agent. A specification with an explicit agent count greater than one, or with per-agent matrix keys (`A_agent1`, `B_agent1`, `C_agent1`, …), declares a multi-agent model: each agent carries its own categorical vocabulary, and coordination is expressed in the `Connections` section rather than by merging the agents into one joint table. `input/gnn_files/multiagent/stigmergic_swarm.md` is the worked exemplar — three agents coordinating through environmental traces (stigmergy) on a shared grid, with no direct communication between them — alongside `multi_agent_coordination.md` and a compact clustered mean-field acceptance fixture.

Hierarchical. Per-level matrix keys (`A_level1`, `B_level1`, `A_level2`, …) declare a multi-level model in which one level's beliefs supply another level's context — `input/gnn_files/hierarchical/hierarchical_pomdp.md` couples a fast lower level to a slow higher level that switches its context, and `temporal_hierarchy.md` develops the same structure over time. These fixtures explicitly declare `block_reset_v1` and `timed_soft_controller_v1`. JAX preserves their block resets and approximate 10/100-clock controller semantics; other backends report these contracts unsupported. Generic hierarchical strategies without these contracts retain their separate capability boundaries.

Learning and factored. A specification that declares a Dirichlet prior over one of its matrices (`dirichlet_A`, …) — or declares a learning `## GNNSection` value — is the parameter-learning kind: a matrix becomes a latent variable inferred from observations rather than a fixed table. `input/gnn_files/learning/dirichlet_likelihood_learning.md` is the worked exemplar: its `StateSpaceBlock` still declares `A[3,3]` in the categorical vocabulary, but `A` is a latent variable with prior pseudo-counts declared in `dirichlet_A[3,3]`, and hidden-state inference and likelihood learning run jointly by variational message passing with a mean-field factorization. The file is explicit about what its `InitialParameterization` means — the numeric `A` there is the environment's ground-truth likelihood that the agent never observes and must recover, not the agent's belief — and its annotation records why the Dirichlet prior is identity-biased rather than uniform: a fully uniform prior leaves the column-permutation symmetry unbroken and inference converges to a label-switched optimum. A specification whose `## ModelParameters` declare more than one independent state factor without per-level or per-agent keys is the factored kind, the standard multi-factor categorical POMDP.

Structural. A file that declares boundary structure only — a Markov-blanket-style wrapper with neither a categorical `A/B/C/D[/E]` parameterization nor a continuous `F/H/Q/R` one — is classified structural and is informational, with no renderable model body: it names interfaces the surrounding system can reason about, and every framework reports it `unsupported` with the reason that it has no renderable form, rather than being forced through a discrete render path that would fail on missing matrices.

The recursive corpus. The `input/gnn_files/recursive/` directory is reserved for bounded `--autonomous` proposal-loop runs and holds no committed models; recursion today enters the taxonomy as structure — a level referencing its own kind, as in the hierarchical compositions above — rather than as a separately renderable kind. The corpus directory is part of the tree so that autonomous runs have a bounded, inspectable home, and its emptiness is a fact of the current release rather than an omission of the documentation.

3.2.4 Reading Exemplars by Kind

Each kind is best understood through a specification that exercises it, and the corpus pairs every kind with at least one canonical exemplar. Reading one file per kind side by side makes the notation-block difference — the entire content of the taxonomy — concrete.

The discrete canonical is `input/gnn_files/discrete/actinf_pomdp_agent.md`, a fully controllable one-factor POMDP. Its `StateSpaceBlock` declares `A[3,3]`, `B[3,3,3]`, `C[3]`, `D[3]`, and `E[3]` — one observation modality, one hidden-state factor, and three actions — and every tensor of eq. 1 through eq. 8 is present by name: `A` carries the likelihood mapping, `B` carries the controlled transitions in the canonical column-stochastic slice order, `C` holds log-preferences, `D` the initial-state prior, and `E` the habit prior. The `Connections` section then wires the inference cycle explicitly — `D>s` conditions the initial belief, `s-A` ties states to observations, `C>G` and `E>`π feed preference and habit into policy scoring, and ``$\pi$>u`` closes the loop from policy to action — so the file reads as a literal transcription of eq. 6 and eq. 7 into declared variables and edges.

The continuous canonical is `input/gnn_files/continuous/continuous_navigation.md` (eq. 9 through eq. 11). Its `StateSpaceBlock` declares no categorical tensors at all: `x[2,1]` and `y[2,1]` are the Gaussian state and observation, `F[2,2]`, `H[2,2]`, `Q[2,2]`, and `R[2,2]` carry the linear dynamics and noise, `prior_mean[2]` and `prior_cov[2,2]` fix the initial belief, and `goal_mean[2]` and `control_gain[1]` add the closed-loop control structure. The `InitialParameterization` gives the matrices numerically — identity dynamics and identity readout, diagonal process and observation noise, a prior centered on the origin, a preferred position, and the proportional gain — and the `Equations` section restates eq. 9, eq. 10, and eq. 11 as the model's declared dynamics. Nothing about the file is a discretized stand-in for a POMDP; it is a linear-Gaussian model by declaration, which is what lets the classifier label it `CONTINUOUS` and the continuous-capable backends render it natively.

The composed kinds show their structure in their key names. `input/gnn_files/multiagent/stigmergic_swarm.md` declares `A_agent1[4,9]`, `B_agent1[9,9,4]`, `C_agent1[4]`, and `D_agent1[9]`, then repeats the block for each further agent, so the per-agent pattern that the classifier keys on is visible directly in the `StateSpaceBlock`; coordination between the agents is expressed through shared environment variables in `Connections`, not by merging the agents into one joint table. `input/gnn_files/hierarchical/hierarchical_pomdp.md` declares `A_level1[4,4]` and `A_level2[4,2]` with matching per-level transitions and priors — the `block_reset_v1` execution contract preserves those levels and resets through JAX rather than flattening this fixture or routing it through RxInfer.jl. In every case the classification is a mechanical reading of the declared names, which is the property that keeps the taxonomy honest: a file is its kind because of what it declares, not because of where it sits.

3.2.5 What the Kind Classification Governs

The classification is not an annotation a report displays and forgets; it governs observable pipeline behavior at each stage that touches a model. At the render step, the per-framework receipts group outcomes into three disjoint sets — frameworks processed, frameworks failed, and frameworks unsupported — and the unsupported set is excluded from the success denominator rather than counted against it: a categorical backend facing a continuous model, or any backend facing a structural wrapper, reports a status with a reason and drops out of the rate, so a pipeline run over a mixed-kind corpus neither inflates its failures nor hides its gaps. At the execute step the same discipline applies, with skipped and degraded lanes recorded as

explicit statuses. At the analysis and reporting steps, the receipts keep the distinctions legible: a continuous model whose categorical-only lanes report `unsupported` by capability reads differently from a discrete model whose continuous attempt failed, and the per-family acceptance ledger that the model-family gate writes carries those statuses per family, per backend, per step.

The governance runs in the direction the taxonomy promises: classification is computed from the parsed specification's declared fields, and composition-aware adapters must preserve the whole kind set. Source-bound render receipts, execution traces, and acceptance ledgers make their handling of those kinds auditable. A reader auditing any of those artifacts can recompute the classification from the file's declared vocabulary alone — tbl. 1 is the key — and expect to match what the pipeline recorded, which is what makes the kind taxonomy a checkable claim rather than a documentation convention.

3.3 The Processing Pipeline

The pipeline is a fixed sequence of 25 numbered steps, 0–24, each a self-contained stage that consumes the artifacts of its predecessors and writes typed outputs for those that follow. Early steps parse and type-check the GNN text and validate it against the language schema; middle steps render visualizations, export the model to executable backends, and run simulations; later steps perform analysis, reporting, and downstream integration. The declared orchestration prerequisites among the steps form the directed acyclic graph shown in fig. 2. The graph records which steps must precede others; artifact receipts separately identify source inputs and produced files, including optional enrichments.

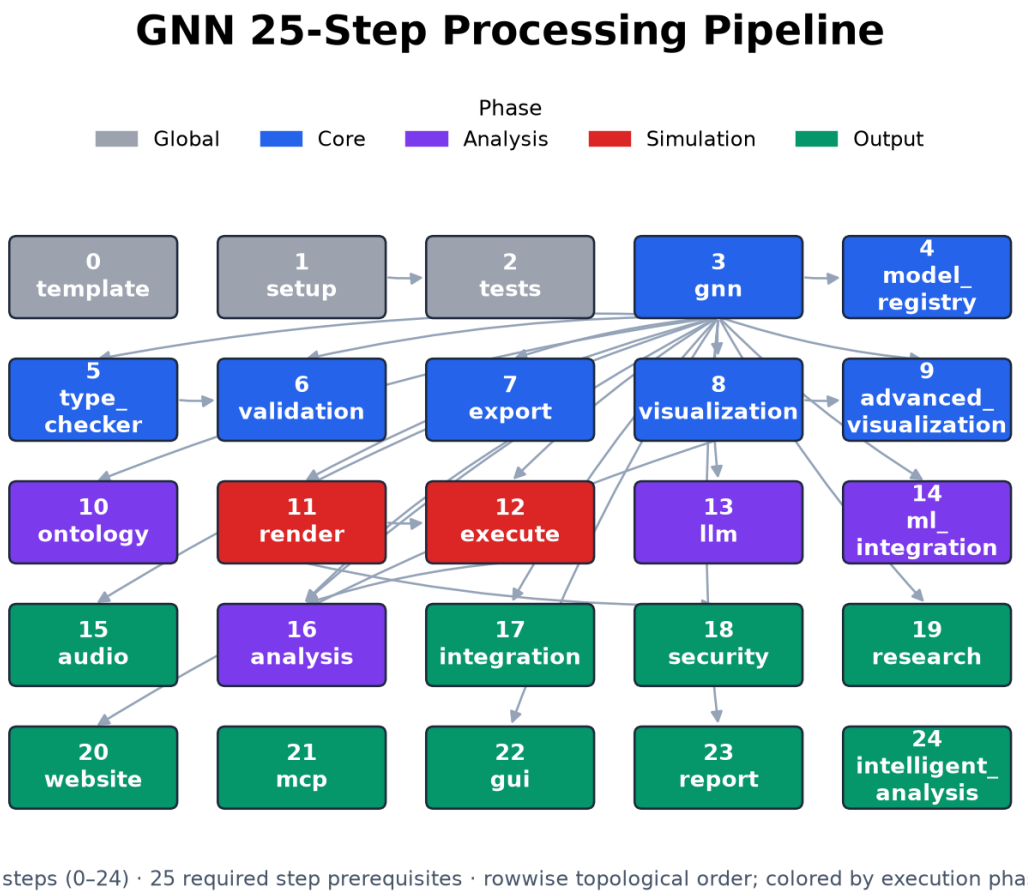


Figure 2: The 25-step GNN processing pipeline as a directed acyclic graph, arranged in rowwise topological order and colored by execution phase (legend above the nodes). Each node is the thin step orchestrator under `src/gnn/` named in tbl. 2; each arrow is a required orchestration prerequisite parsed from the Data Dependency Graph block of `src/gnn/STEP_INDEX.md`, which mirrors the live step registry. Parsing (Step 3) precedes many later steps. Optional artifact enrichments are documented separately and are not shown as required edges. The figure is generated from `src/gnn/STEP_INDEX.md` at commit d20b76a54; its footer restates the step count, the number of required prerequisites, and the layout rule.

The per-step responsibilities are enumerated in tbl. 2; each row names a step and the transformation it owns within the 0–24 range.

Table 2: The pipeline steps with the thin orchestrator module that owns each and the one-line purpose parsed from the master table in `src/gnn/STEP_INDEX.md`. Read the Step column against the arrows of fig. 2: each row's purpose is the transformation that module owns, and the table is regenerated from the index at every build, so it cannot drift from the code it describes.

Step	Module	Purpose
0	`0_template.py`	Pipeline template & initialization
1	`1_setup.py`	Environment setup & UV dependency install
2	`2_tests.py`	Test suite execution (pytest)
3	`3_gnn.py`	GNN file discovery & multi-format parsing
4	`4_model_registry.py`	Model versioning & registry management
5	`5_type_checker.py`	GNN type validation & resource estimation
6	`6_validation.py`	Consistency & semantic quality checking
7	`7_export.py`	Multi-format export (JSON, XML, GraphML, GEXF, Pickle)
8	`8_visualization.py`	Graph & matrix visualization generation
9	`9_advanced_viz.py`	Interactive / advanced visualization (Plotly, D3)
10	`10_ontology.py`	Active Inference ontology processing & validation
11	`11_render.py`	Code generation for simulation frameworks
12	`12_execute.py`	Execute rendered simulation scripts
13	`13_llm.py`	LLM-enhanced analysis & model interpretation
14	`14_ml_integration.py`	Machine learning integration & model training
15	`15_audio.py`	Audio sonification generation (SAPF)
16	`16_analysis.py`	Statistical analysis & cross-simulation aggregation
17	`17_integration.py`	System integration & cross-module coordination
18	`18_security.py`	Security validation & generated code scanning
19	`19_research.py`	Research tools & literature references
20	`20_website.py`	Static HTML website generation
21	`21_mcp.py`	Model Context Protocol processing & tool registration
22	`22_gui.py`	Interactive GNN constructor GUI
23	`23_report.py`	Comprehensive analysis report generation
24	`24_intelligent_analysis.py`	AI-powered pipeline analysis & executive reports

This staged design keeps the architecture modular. The implementation is organized into 44 source packages, one cluster of responsibilities per concern, and is documented across 641 documentation files so that each step's contract, inputs, and outputs are specified independently of the others. New backends or analyses attach to the graph by declaring their dependencies rather than by editing a monolith, and deterministic step ordering makes the processing sequence reproducible. Artifact receipts bind each run to its selected model sources and resolved configuration; numerical replay also depends on the backend, dependency versions, dtype, and random seed, while run identifiers and timestamps may differ. The pipeline is kind-agnostic in its plumbing and kind-aware at its edges: parsing, validation, and export operate on whatever the file declares, while the rendering and execution steps consult the classification of sec. 3.2 to decide which backends a given model can reach and

record, explicitly, the ones it cannot.

3.4 The Triple Play

The reason for separating a single written language from a multi-stage pipeline is the design goal GNN calls the Triple Play: one model specification, three coordinated modes of existence. The same GNN text supplies a human-readable model description, the declared state-space and factor structure for graphical visualization, and input to supported executable renderers. Source identity and validation receipts connect these representations to the authored model; runtime readiness and model-kind admission govern which simulations can execute. fig. 3 depicts these three faces and the shared specification at their center.

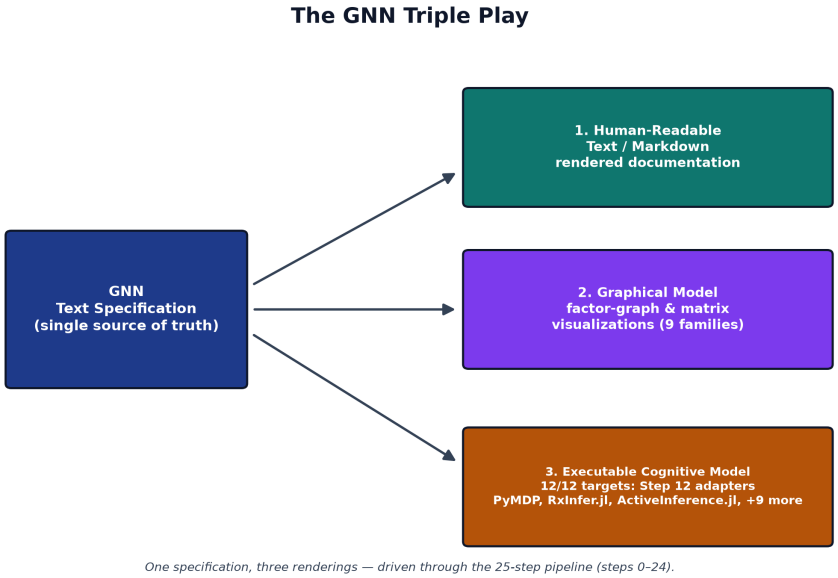


Figure 3: The Triple Play: one GNN text specification (left) and its three coordinated projections (right) — the human-readable text form, graphical model visualizations, and executable simulation code. All three arrows originate at the same source box to show their shared parsed input. Source binding, parameter validation, and semantic-fidelity checks provide evidence of agreement among the projections. The executable node summarizes 12 of 12 registered render targets with implemented Step 12 execution adapters and previews three by name; execution depends on runtime readiness, admitted model kind, and a successful source-bound run receipt. Counts are producer tokens at commit d20b76a54, and the panel is generated by `scripts/manuscript_fig_triple_play.py`. Take-away: a model is authored once and then read, seen, and run from that single source.

Each face derives from a shared authored source. The text is the contract a researcher reads and reviews; the visualizations expose the declared factor-graph structure for inspection and communication; and the executable rendering targets inference frameworks, connecting the written specification to the discrete message-passing libraries for categorical Active Inference [Heins et al., 2022], to the reactive message-passing and probabilistic-programming engines that carry the linear-Gaussian kind, and to the broader tooling ecosystem for compositional model representation [de Felice et al., 2021]. Common source identity makes the projections traceable, while parameter values, tensor orientation, model-kind semantics, and source-bound numerical results still require validation to establish agreement. A supported model can therefore be read, seen, and run without authoring a separate specification for each purpose [Smith et al., 2022]. The Triple Play is governed per kind: renderers and executors apply the computed classification and their declared capability boundaries, and receipts distinguish successful, unsupported, skipped, partial, and failed outcomes for the selected model.

4 Methods

The GeneralizedNotationNotation (GNN) method carries a plain-text model specification through source-processing transformations, bounded execution, and reporting. Parsing and code generation can be deterministic for fixed inputs and configuration; inference may involve stochastic sampling and depends on the runtime, dtype, and seed. The processing pipeline is organized into 25 numbered steps (steps 0–24), each implemented as a standalone module with a single responsibility. The methods below describe the path that a model travels from notation to executable cognitive model and back to analyzed results, and they state that path per model kind wherever the kinds part ways: the pipeline's plumbing is kind-agnostic, but rendering and execution are kind-aware by design. Every quantity reported in this section is produced from the live repository rather than asserted by hand; the closing subsection makes that contract explicit.

4.1 Parsing and Multi-Format Export

The pipeline begins by ingesting GNN model files written in the plain-text notation. Parsing (step 3) reads each specification, builds an internal model representation, and re-emits it across a family of structured export formats so that downstream tools, and human readers, can consume the same model through whichever serialization they prefer. The corpus that exercises this stage spans 38 example model files organized into 12 curated corpora, and it spans the model kinds of sec. 3.2: minimal perception fixtures used to test the parser, discrete categorical agents, continuous linear-Gaussian state-space models, multi-agent swarms, hierarchical two-level compositions, and the scaling-study corpus that grows a single family across state-space sizes. The presence of hierarchical fixtures should be read as a coverage target, not as a claim that every current backend fully executes every hierarchical formulation; scaling hierarchical active inference remains an active research problem in its own right [Rangarajan and Rao, 2026]. Treating parsing and export as a single round-trippable stage means a model authored once becomes immediately available as a typed object, a normalized text form, and machine-readable serializations without any manual re-encoding — and because the typed object carries the declared matrix vocabulary verbatim, the kind classification of sec. 3.2 is computed from the parse result rather than re-derived downstream.

4.2 Type Checking and Validation

A model that parses is not yet a model that is well-formed. Steps 5 and 6 apply type checking and validation to confirm that the state spaces, observation modalities, control factors, and the matrices relating them are mutually consistent before any code is generated. This catches dimensional mismatches and malformed factor structures at the notation level, where the diagnostic is cheap and legible, rather than allowing them to surface as opaque runtime errors deep inside a numerical backend. Validation is where the kinds receive their shape contracts, and each kind's contract checks the objects its notation actually declares.

For a discrete categorical specification, validation reads each declared `B` tensor through an orientation diagnostic: the canonical reading is column-stochastic, `B[next_state][previous_state][action]`, matching the pymdp convention [Heins et al., 2022] and the transition semantics fixed by eq. 3, and the check records every tensor's detected orientation in its receipt. A row-stochastic textbook literal — the transpose of the canonical layout — is flagged with the tensor, state factor, and flipped slice indices named, and the opt-in `--transpose-b` flag maps such literals onto the canonical order at load time instead of failing them, recording the transposition per tensor.

For a continuous linear-Gaussian specification, validation enforces the Gaussian shape contract of eq. 9 and eq. 10: `prior_mean` must carry exactly as many entries as the latent state, `Q` and `prior_cov` must be square in the state dimension, and `R` must be square in the observation dimension. A covariance whose shape disagrees with the state or observation declaration is a validation-time diagnostic naming the mismatched matrix and its dimensions, not a broadcast error inside a generated script; and the optional closed-loop declarations are checked as a pair, since a `goal_mean` without a `control_gain` (or the reverse) is an incomplete control structure rather than a passive model.

Validation supplies explicit checks of the declared model contract before rendering. Passing those checks establishes their scoped conditions, not a certificate of every later numerical or backend behavior; execution and analysis retain their own validation and source-bound receipts.

4.3 Rendering to Multiple Backends

The central act of the "Triple Play" is turning a validated specification into executable cognitive models. The rendering stage emits backend-specific code for 12 registered target frameworks — PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn — so that a single GNN model can be instantiated as a simulation in whichever computational ecosystem a researcher already works in. Each backend is addressed through a registry key that maps the abstract model onto that framework's idioms for representing generative models and performing inference. The full mapping from registry key to backend is given below.

Table 3: Render targets declared in `src/gnn/render/framework_registry.py`, one row per registry key in registry order. The *Executes* column is the registry's own `supports_execution` flag: 12 of 12 backends have a Step-12 executor, and a `render-only` entry would mean generated code with no Step-12 executor. Read this table against tbl. 5, which shows which families actually target each backend.

Registry key	Backend	Executes
`pymdp`	PyMDP	yes
`rxinfer`	RxInfer.jl	yes
`activeinference_jl`	ActiveInference.jl	yes
`jax`	JAX	yes
`discopy`	DisCoPy	yes
`pytorch`	PyTorch	yes
`numpyro`	NumPyro	yes
`stan`	Stan	yes
`bnlearn`	bnlearn	yes
`cpomdp`	cpomdp	yes
`thrml`	THRML	yes
`ngclearn`	ngc-learn	yes

Because rendering is kind-aware, the registry is best read as a capability matrix rather than a flat list: which backends can render a discrete categorical model, which can render a continuous linear-Gaussian model, and which have execution adapters at Step 12. That per-framework, per-kind view is auto-injected below.

Table 4: Model kinds each render framework supports, read entirely from the flags in `src/gnn/render/framework_registry.py` — the same source as tbl. 3. A continuous cell of `unsupported` is the renderer's own status for continuous-state models; continuous-only targets do not render categorical models. An executor entry declares dispatch support, not acceptance of every model or inference mode; a render-only entry has no Step-12 executor. Read with tbl. 1, which groups the same flags by model kind.

Framework	Discrete render	Continuous render	Executor status
PyMDP	yes	unsupported	executor
RxInfer.jl	yes	yes	executor
ActiveInference.jl	yes	unsupported	executor
JAX	yes	yes	executor
DisCoPy	yes	unsupported	executor
PyTorch	yes	yes	executor
NumPyro	yes	yes	executor
Stan	yes	yes	executor
bnlearn	yes	unsupported	executor
cpomdp	no	yes	executor
THRML	yes	unsupported	executor
ngc-learn	no	yes	executor

The registry surface of these backends — registry key, display name, implementation language, and which of them the cross-framework gate compares — is summarized in fig. 4; the family-by-backend coverage itself is shown in fig. 6.

4.3.1 Rendering by Model Kind

Discrete categorical models use the categorical-capable renderers, subject to each renderer's dimension, parameter, and composition checks. The continuous-only cpomdp and ngc-learn targets report these models as `unsupported`. Experimental THRML must be selected explicitly: it implements fixed-action categorical posterior sampling, with bounded independent or validated joint compositions, and does not optimize policies or expected free energy. Registry membership therefore does not imply categorical support for every target or every categorical composition; the profiled cross-framework reference in sec. 5 is a separate, selected comparison.

Continuous linear-Gaussian models take the native paths built for them. The RxInfer.jl renderer carries dedicated continuous strategies that emit reactive message-passing code over the Gaussian state-space model; the JAX, PyTorch, and NumPyro renderers generate standalone scripts that filter the belief state numerically — and, when the closed-loop declarations of eq. 11 are present, steer it toward the preferred state — with the JAX lane enabling 64-bit precision for the covariance arithmetic; and the Stan renderer emits the model as a probabilistic program. The categorical-only backends — pymdp, ActiveInference.jl, DisCoPy, and bnlearn — do not fabricate a translation: each reports the continuous model

GNN Rendering Backend Registry
cross-framework reference family: continuous

Registry Key	Display Name	Language	Render Subdir	Cross-Framework
pymdp	PyMDP	Python	yes	
rxinfer	RxInfer.jl	Julia	yes	profiled
activeinference_jl	ActiveInference.jl	Julia	yes	
jax	JAX	Python	yes	profiled
discopy	DisCoPy	Python	yes	
pytorch	PyTorch	Python	yes	
numpyro	NumPyro	Python	yes	profiled
stan	Stan	Stan	yes	declared
bnlearn	bnlearn	Python	yes	
cpomdp	cpomdp	Python	yes	
thrml	THRML	Python	yes	
ngclearn	ngc-learn	Python	yes	declared

12 registered backends · 12 with a render-output subdir under src/gnn/render/ · highlighted 3 backends are profiled in the continuous reference comparison
stan, ngclearn declared by the family but outside MAINTAINED_FRAMEWORKS, so never profiled

Figure 4: The GNN rendering backend registry as a table: one row per registered backend, with its registry key, display name, implementation language, whether a render-output subdirectory exists under `src/gnn/render/`, and its role in the cross-framework reference comparison. Amber rows are the backends the reliability gate profiles for the continuous family; the tan row (Stan, ngc-learn) is declared by that family but sits outside the maintained set, so it is never profiled; a red Render-Subdir cell would mean a registry entry with no subdirectory. The table is read directly from `src/gnn/render/framework_registry.py` and the model-family manifest at commit d20b76a54, and its footer summarizes the row counts. Take-away: registration, render presence, and profiling status are three different claims, and the table keeps them apart.

as `unsupported`, with a recorded reason, and the status is counted under unsupported framework renderings rather than as a failure — never rendered, never executed, and excluded from the success denominator so the run's rate measures the backends that could actually attempt the model. DisCoPy's boundary is representative: its translator draws categorical POMDP string diagrams and has no linear-Gaussian diagram semantics, so a continuous model is reported unsupported rather than drawn as a discrete stand-in.

Composed kinds require admission of the whole declared composition, not just one recognized substrate. Supported categorical multi-agent paths map per-agent declarations to their target structures. Independent continuous agents use native JAX or RxInfer.jl paths only when independence and a complete linear-Gaussian parameter block for every agent are declared; other continuous-agent compositions are reported `unsupported`. Generic hierarchical paths without a semantic execution contract may compose per-level matrices into a joint POMDP or use the RxInfer.jl two-level strategies; the maintained `block_reset_v1` and `timed_soft_controller_v1` fixtures instead route to structure-preserving JAX execution and are explicitly unsupported elsewhere; parameter-learning paths use strategies for the declared Dirichlet matrix. These paths retain backend-specific capability boundaries, and a recognized component does not authorize silently dropping another declared component.

Structural wrappers have no renderable form at all: they declare boundary structure only, with neither a categorical nor a continuous parameterization, so every framework reports them `unsupported` upstream with the structural-spec reason rather than silently rendering them as a discrete stand-in that would fail on missing matrices. The status is the design working, not a defect: a wrapper is an interface statement, and the pipeline's response is to say so in every receipt rather than to guess a model body the author never declared.

By generating code rather than asking authors to port models by hand, the method keeps a single source of truth in the notation while still reaching the discrete message-passing libraries [Heins et al., 2022], reactive message-passing lineage [Bagaev and de Vries, 2021], and categorical, diagrammatic frameworks [de Felice et al., 2021] that different research communities have built. The underlying mathematics that compatible backends share — inference over the generative model the notation declares, whether that is categorical free-energy minimization [Da Costa et al., 2020, Smith et al., 2022] or Gaussian filtering and control — is the common substrate the renderer attempts to preserve; where a backend cannot express a kind faithfully, GNN records that boundary as an explicit unsupported status rather than treating all registered backends as equivalent.

4.4 Execution

4.4.1 Executing by Model Kind

Generated backend code is not left as a static artifact. The execution stage (step 12) runs the rendered models, driving the inference and behavior that the notation describes and producing concrete traces, beliefs, and outputs. The 12 render targets — PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn — have implemented execution adapters at this step. This is a registry claim, not evidence that every adapter has run in a given environment. Model-kind admission, validated parameters, package and toolchain readiness, supported Python, and remaining budgets determine whether an execution can proceed; the current-run receipt records its actual outcome.

Discrete executors run the categorical models end to end, and they guard their own runtime assumptions: the generated pymdp scripts, for example, sanity-check the installed library for the current Agent interface at startup and exit fast with a clear message when an unsupported wheel is present, so a stale environment produces a legible diagnostic instead of a mysterious failure deep in a simulation loop.

Continuous executors run the generated standalone scripts over the Gaussian state-space model — filtering to a posterior, and with the closed-loop declarations in place, closing the control loop on beliefs — so a continuous model produces traces on the same terms as a discrete one, on the backends that carry the kind.

Generator-backed and render-adjacent lanes extend execution beyond the registry's render paths. The bnlearn lane is generator-backed: it renders a runnable Python program from the same parsed model the other lanes consume, and it skips with a recorded status when the `bnlearn` package is absent, so a lean checkout degrades explicitly rather than failing. A separate execute-side bridge verifies rendered Lean documents against the fep_lean bridge contract, which is how the notation reaches a proof assistant without a render-registry entry. Skipped, degraded, and unsupported lanes are recorded as explicit statuses in the cross-framework comparison rather than silently omitted.

For a successfully executed, source-bound model, this closes the loop from text to numerical traces. Claims about its behavior require the actual result, inference mode, environment, and validation receipts; a generated script or implemented adapter alone does not establish that behavior.

4.5 Long-Running Orchestration Contracts

GNN 3.0.0 extended the pipeline with three safe-by-design orchestration contracts that let an extended run be observed, paused, resumed, and audited without ever mutating live infrastructure. *Durable observation streams* record file- and array-backed stream manifests with content checksums and replayable execution traces, so that a long run can be re-derived

deterministically before any live sensor or device-backed stream is introduced. *Resumable run sessions* carry run-session manifests with atomic checkpoint and resume, status inspection, and cancellation-safe cleanup, so that an extended model-family acceptance run can be interrupted and resumed without corrupting partial state. *Auditable container plans* generate hardened container plans together with an explicit static security review and a rollback descriptor, deliberately stopping short of touching any real cluster. Each contract only generates and validates data; none performs live mutation, and each is governed by a strict acceptance gate and exercised through dedicated Model Context Protocol tools. fig. 5 shows how the three contracts compose into one inspectable orchestration surface.

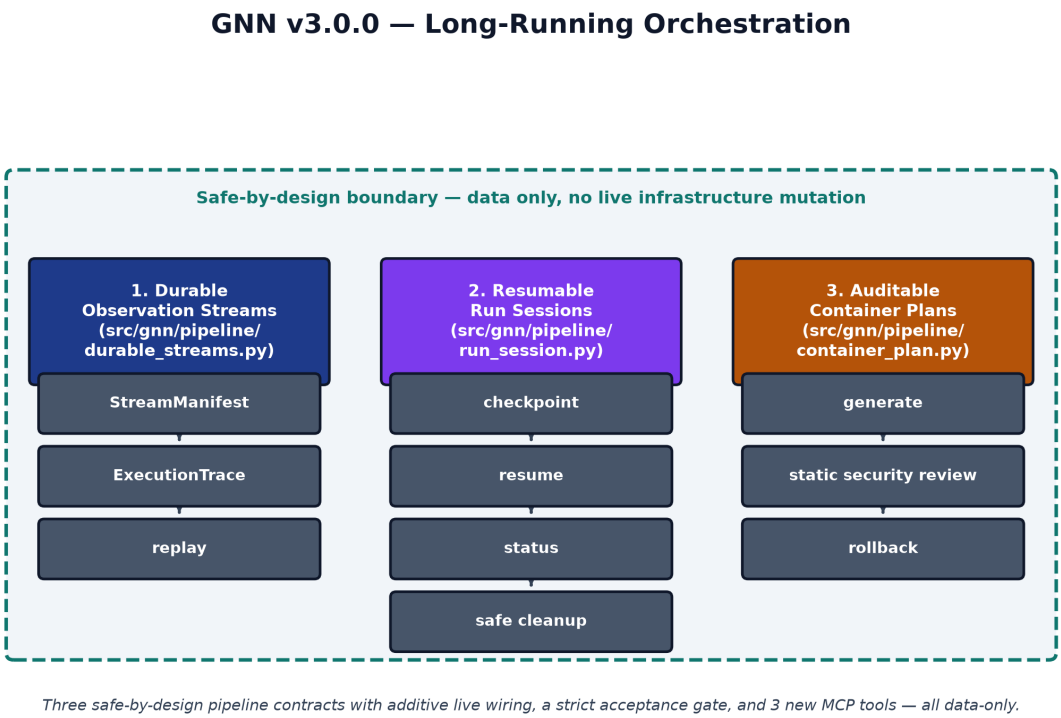


Figure 5: The three safe-by-design orchestration contracts introduced in GNN 3.0.0, side by side inside the dashed boundary that states their shared invariant: generate and validate data only, never mutate live infrastructure. Each column names the contract's source module under `src/gnn/pipeline/` and chains its stages top to bottom — StreamManifest, ExecutionTrace, and replay for durable observation streams; checkpoint, resume, status, and safe cleanup for resumable run sessions; generate, static security review, and rollback for container plans. Arrows give the order the stages run in, and nothing exits the boundary. The panel is a fixed-layout drawing of the three contracts; the acceptance gates and the Model Context Protocol tools that exercise them are described in the text above. Take-away: long-running orchestration gains capability only as fast as it becomes inspectable.

4.6 Analysis, Visualization, and Reporting

The final group of stages turns execution results into interpretable evidence. Analysis (step 16) processes the outputs of execution into structured findings; visualization (step 8) and the rendering of figures (step 9) translate model structure and results into graphical form, supporting the graphical leg of the Triple Play; and the reporting stage (step 23) assembles these artifacts into a coherent summary of what the model is and how it behaved. Known output locations and current-run identity indexes connect a notation file to its report artifacts. Tracing a figure or number requires its selected model identity and source, script, result, and artifact receipts; an output path alone is insufficient evidence of a current run. Analysis reads the per-kind rendering and execution receipts as well: a continuous model whose categorical-only lanes report `unsupported` yields a different evidence profile than one whose continuous-capable lanes failed, and the reports keep those situations apart.

4.7 Reproducibility and Auto-Injection

Every count reported in this manuscript is emitted by the producer `scripts/z_generate_manuscript_variables.py` from the repository state at commit d20b76a54, not typed by hand. The producer reads that commit's source surfaces directly — the step modules, the source tree, the model-family manifest, and the framework registry — together with the project's maintained ledgers, notably the Model Context Protocol tool audit (`src/gnn/mcp/audit_report.json`, itself regenerated by the test suite). From these it emits a manifest of named tokens — counts of pipeline steps and their modules, source packages and files, model families, backends, example files partitioned by model kind, tests, and tools — plus the two capability tables

this manuscript injects: the model-kind table (tbl. 1) and the framework capability matrix (tbl. 4). The renderer substitutes the tokens into the prose at build time. Filesystem-derived counts are recomputed on every run; ledger-derived counts (such as the backend registry) are re-read from a fixed repository state, so a fixed repository state yields the same values, and authors are prohibited from hard-coding any number that has a corresponding token. This makes the manuscript a faithful, regenerable description of the system it documents: when the 25-step pipeline, the 12 rendering backends, or the 38-file example corpus change — or when the discrete-to-continuous exemplar balance shifts — the reported numbers change with them on the next build, and `scripts/check_manuscript_tokens.py` fails the build if a section reintroduces a literal that a token already owns.

5 Artifacts and Evidence

This section reports what the project has actually produced and how each quantitative claim is grounded. Every number below is substituted at render time from the deterministic producer, which reads the repository state at commit d20b76a54, so each figure here is regenerated from the artifacts it describes rather than transcribed.

5.1 Model-Family Coverage

GNN ships a curated corpus of model families that exercise the language across the difficulty gradient from minimal parser fixtures to full multi-agent and scaling studies. The manifest registers 9 families (basics, discrete, continuous, hierarchical, multiagent, precision, structured, gridworld, scaling-study) across 9 target directories, and `input/gnn_files` holds 12 corpus directories containing 38 concrete example models. The three sets are not coextensive, and the manuscript does not treat them as one. Registered in no family: `input/gnn_files/learning`, `input/gnn_files/recursive`, `input/gnn_files/thrml` (3 of the 12 corpus directories), covered in sec. 7. Registered but outside the scanned tree: none, because all 9 target directories are themselves `input/gnn_files` corpus directories. Each family declares the simulation frameworks it is meant to drive, which is what lets the same text model fan out across the executable-model leg of the Triple Play [Smékal and Friedman, 2023]. The families and their declared frameworks are enumerated below.

The exemplar corpus partitions cleanly by model kind, and the partition is itself a token: 29 of the 38 example specifications are discrete-state — the discrete categorical kind and its composed variants — and 9 are continuous linear-Gaussian specifications in the continuous family folder. The cold-start index `input/gnn_files/INDEX.md` catalogues every exemplar with its kind, its task folder, and a suggested reading order, and is the fastest surface for a reader who wants to move from this prose to a runnable file of a given kind.

Families that declare a capability axis (the continuous family declares the continuous-render axis) carry a backend split in tbl. 5 generated directly from the registry's declared capability flags. This avoids a manually copied registry table; actual model admission still depends on the full parameter and composition checks recorded by the renderer. The generated flags describe declared capability, while current-run receipts record which selected backends were actually attempted, unsupported, skipped, or failed. The profiled cross-framework reference uses the continuous family, so it does not establish categorical coverage across the registry (tbl. 4 is the per-kind key).

Table 5: Model families declared in `input/model_family_manifest.json` with the simulation frameworks each family targets and a generated description. The *Frameworks* column is the family's declared targets, verbatim from the manifest; the capability splits appended to some descriptions are generated from `src/gnn/render/framework_registry.py` flags, not authored in the manifest, so they cannot disagree with the registry. Read with tbl. 3 (what each backend is) and fig. 6 (the same coverage as a matrix).

Family	Frameworks	Description
`basics`	pymdp	Minimal perception fixtures used for parser and validator smoke coverage.
`discrete`	pymdp, jax	Discrete POMDP, passive HMM filtering and episodic contingent T-maze fixtures.
`continuous`	jax, numpyro, stan, rxinfer, ngclearn	Continuous-state linear-Gaussian fixtures (F/H/Q/R + Gaussian prior; continuous_navigation closes the loop on beliefs; ngc-learn LGSSM exemplar for the ngclearn backend). Native on RxInfer.jl, JAX, PyTorch, NumPyro, Stan, cpomdp, ngc-learn; reported unsupported (not failed) on PyMDP, ActiveInference.jl, DisCoPy, bnlearn, THRML.
`hierarchical`	jax	Declared block-reset hierarchy and approximate 10/100-clock soft-message controller fixtures.
`multiagent`	rxinfer	Multi-agent coordination and swarm fixtures.
`precision`	pymdp	Precision weighting and curiosity-driven fixtures.
`structured`	pymdp	Structured factor graph and posterior fixtures.

Family	Frameworks	Description
`gridworld`	pymdp, rxinfer, activeinference_jl	Gridworld POMDP fixture used for cross-framework acceptance checks.
`scaling-study`	pymdp	PyMDP scaling-study fixtures, sampled conservatively for acceptance.

The family-by-framework structure is shown in fig. 6, which renders the coverage matrix directly from the family registry rather than from a hand-maintained table.

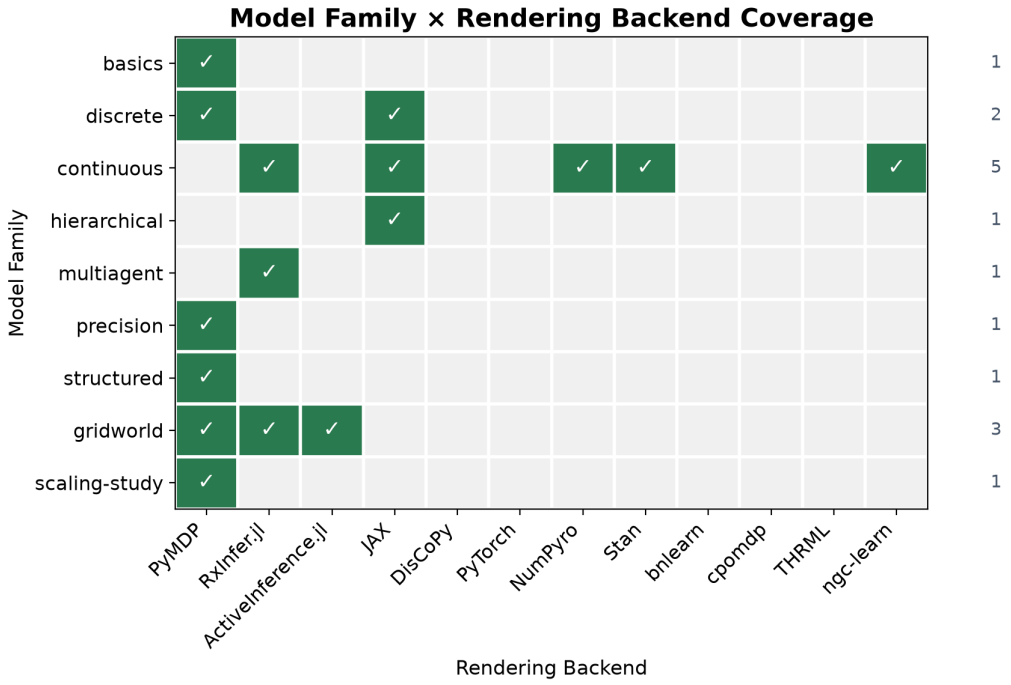


Figure 6: Model-family coverage across the registered rendering backends: one row per family from `input/model_family_manifest.json`, one column per backend from `src/gnn/render/framework_registry.py`, and a green cell wherever the family declares that backend in its `frameworks` field. The right-hand count states how many backends each family declares; the grid is deliberately sparse — most families declare a single backend, and only continuous, discrete, and gridworld declare several. Read it as declared intent, not as profiled outcomes: the gates described below supply the outcomes. The matrix is generated from the two registries at commit d20b76a54.

These families are not illustrative prose: they are the inputs over which the parser, the type checker, and the cross-framework code generators are exercised, and they are the substrate for the reliability gates described next.

5.2 Semantic-Fidelity and Cross-Framework Reliability Gates

Two command-defined gates check distinct parts of GNN's shared-source contract on selected family fixtures. Both live under `scripts/` and use the family manifest described above. Their actual source-bound command receipts determine the scope and outcome of verification; the presence of a gate is not itself a passing result.

The semantic-fidelity gate, `scripts/run_semantic_fidelity_gate.py`, checks parse–serialize–parse preservation using the versioned `gnn_semantic_contract_v2` contract. It compares model identity, variables, edges, parameter names, shapes, values and metadata, equations, timing, and ontology mappings; the default strict round trip is JSON. Parameter values and their axis order contribute to the contract hash, so changing values without changing shape changes the hash. The digest binds the represented contract; it does not prove that extraction was correct, that generated inference implements the intended mathematics [Da Costa et al., 2020], or that independent numerical executions agree. The gate reports scoped fidelity per selected model and format.

The cross-framework reliability gate, `scripts/run_cross_framework_reliability.py`, takes a single GNN model and renders it across multiple simulation backends, then checks that the resulting executable models agree on the structure they were

generated from. The reference comparison runs on the continuous family across JAX, NumPyro, RxInfer.jl — 3 independent Active Inference engines spanning the Python and Julia ecosystems [Heins et al., 2022]. The family declares 5 backends (JAX, NumPyro, Stan, RxInfer.jl, ngc-learn); Stan, ngc-learn is declared but not among the 7 frameworks the gate profiles, so it is excluded from the comparison. Because the same source model drives all 3 renderings, source identity helps investigate disagreement. The gate compares shared matrix shapes, trace lengths, timesteps, and reported seeds, recording absent comparable fields separately. These checks do not establish numerical equivalence: a mismatch may arise from extraction, code generation, runtime or toolchain behavior, inference mode, dtype, or a backend defect. Numerical comparisons require compatible models and inference semantics, explicit tolerances, and source-bound numerical witnesses.

Both gates are stated here as commands you can run, not as asserted pass counts. The manuscript deliberately does not quote a fixed number of passing checks: an authoritative result is a retained command receipt bound to its source revision, selected corpus, environment, and artifacts, and binding a frozen count into prose would invite exactly the drift the auto-injection contract exists to prevent.

A third interchange check extends the same discipline across repositories: `scripts/run_geo_interchange_checks.py` validates the committed pin in `.github/gnn-pair.json` against a selected GEO-INFER checkout and replays exported GNN artifacts — the tracked gridworld, a compiled H3 stay/diffuse model, a rectangular Gaussian, and an explicit factored fixture — inside the GEO environment, writing its receipts even on failure. Like the gates above it is stated as a command, not as a pass count, and its pinned pair (`ActiveInferenceInstitute/GEO_INFER` at a recorded revision) is what makes "interchange" a checkable claim rather than a promise.

5.3 Repository Scale

The repository's scale is itself evidence of the surface that the gates and pipeline cover, and it is reported in fig. 7 directly from the tracked files at commit d20b76a54.

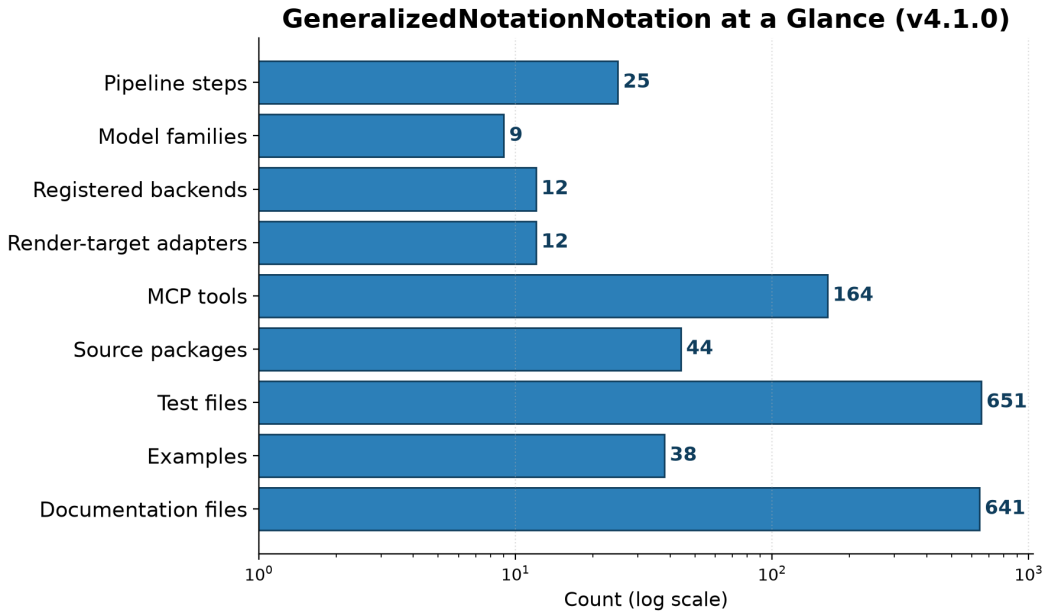


Figure 7: Repository-scale counts on a logarithmic axis: pipeline steps, model families, registered backends, render-target execution adapters, Model Context Protocol tools, source packages, test files, example models, and documentation files. Every bar is annotated with its exact value, and every value is a producer token read from `output/data/manuscript_variables.json` at commit d20b76a54 — the same token map that substitutes the prose counts. The figure-freshness suite binds the committed image digest and its recorded producer inputs and counts; semantic content and visual correctness require their own review. The two backend bars are deliberately distinct: *registered backends* (12) counts render targets, *render-target adapters* (12) counts the render-target subset with implemented Step 12 execution adapters. It excludes the separate execution-only Lean bridge and is not a count of observed native runs. Read the chart as the scale of the surface the pipeline maintains, not as a quality measure.

The test suite comprises 651 test files containing 6694 test functions, exercising a source base of 759 Python files across 44 packages (223521 lines of source). The pipeline's step modules — the thin orchestrators named in tbl. 2 — number 25, one per numbered step. The Model Context Protocol surface — which exposes GNN's capabilities to external agents and tools — provides 164 tools across 36 modules. The pipeline itself runs as 25 steps (0–24), and 7 figure artifacts from the rendering

of figures, models, and reports are committed under `output/`, of which 7 are the manuscript's own.

5.4 Claim Discipline

A claim is manuscript-ready only when it is bound to a verifiable artifact. Concretely, every claim in this manuscript must rest on one of four support types:

- A retained passing test or validator command receipt, bound to the source revision, selected inputs, environment, and artifacts — for example a scoped execution of the semantic-fidelity or cross-framework gates above.
- A generated output produced by a deterministic producer, such as the figures rendered from the family and repository scans, or the token values emitted by the manuscript-variable producer that backs every number on this page.
- A source ledger, manifest, or configuration file that fixes a declared value or capability; actual execution still requires its own result receipt.
- A resolved entry in `references.bib` for any external-literature claim [Friston, 2010, Parr et al., 2022].

The pipeline records its own evidence trail under `output/`. The run-level summary is written to `output/PIPELINE_REPORT.md`, and the per-step execution record — including which steps ran, their status, and their artifacts — is captured in `output/00_pipeline_summary`. Readers should use those current-run records and their identity bindings to assess actual pipeline outcomes. Repository counts and declared capability tokens describe the source surface and do not by themselves establish native execution or scientific correctness. When a value would otherwise need a literal number with no producer behind it, the discipline is to omit the number rather than to hard-code it.

6 Reproducibility

Reproducibility in GNN is a source- and run-binding contract across the 0–24 processing steps. Fixed-source parsing and token production are distinct from stochastic inference, dependency provisioning, and environment-dependent execution. Reproducing a reported outcome requires the selected inputs, configuration, source revision, dependencies, dtype, seed, and retained command and artifact receipts. This section lists only commands that exist in the repository, so that a reader with a clean checkout can reproduce the pipeline, the validation gates, and this manuscript itself.

6.1 Pipeline Smoke Run

The fastest way to confirm a working installation is to drive the full pipeline over the discrete model family without invoking the optional LLM steps:

```
uv run python src/gnn/main.py \
  --target-dir input/gnn_files/discrete \
  --output-dir /tmp/gnn-smoke \
  --skip-llm
```

This parses the discrete GNN files, runs visualization and rendering across the maintained backends, and writes all artifacts under the chosen output directory. The `--skip-llm` flag disables the LLM processing step; it does not establish network isolation or guarantee that every other step executes. Default framework selection excludes the experimental targets, and dependency readiness, model-kind admission, configured steps, and budgets determine the actual work. Inspect the current-run statuses and terminal exit code (0 success, 1 error, 2 warning); missing or unfinished required work must remain a non-success outcome. To exercise every registered family rather than a single one, drive the manifest through the model-family acceptance gate given below: pointing `--target-dir` at `input/gnn_files` covers that tree's 12 corpus directories. All 9 registered family target directories lie inside that tree, so a single invocation reaches every registered family.

The discrete family exercises the categorical kind end to end. The continuous linear-Gaussian kind smoke-runs the same way, and the contrast between the two runs is itself a check of the per-kind contract:

```
uv run python src/gnn/main.py \
  --target-dir input/gnn_files/continuous \
  --output-dir /tmp/gnn-smoke-continuous \
  --skip-llm
```

This selects the continuous specifications for the same pipeline. Admitted, ready continuous-capable lanes can render and execute filtering (and, for the closed-loop exemplar, belief-steering) programs; categorical-only targets record `unsupported`, while readiness and budget failures retain their own outcomes. A reader comparing the two receipts sees the kind taxonomy behaving as described in sec. 3: same pipeline, same steps, per-kind rendering and execution reach.

6.2 Validation Gates

GNN provides strict gates with distinct verification scopes. The model-family acceptance gate runs the selected manifest families and evaluates their declared acceptance profiles; its receipt must record actual execution, unsupported and skipped work, and failures:

```
uv run python scripts/run_model_family_acceptance.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/model_family_acceptance --strict
```

The semantic-fidelity gate verifies that a parse → serialize → parse round trip preserves variables, edges, dimensions, parameter shapes and values, parameter metadata, equations, time semantics, and ontology mappings across the 9 model families; the cross-framework gate profiles the 7 maintained backends (PyMDP, RxInfer.jl, JAX, NumPyro, PyTorch, ActiveInference.jl, DisCoPy) — refusing any framework outside that set — and records explicit compatible and unsupported statuses rather than silently degrading. Both write their ledgers to an output directory of your choosing:

```
uv run python scripts/run_semantic_fidelity_gate.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/semantic_fidelity --strict
uv run python scripts/run_cross_framework_reliability.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/cross_framework --strict
```

Under `--strict`, required mismatches make these gates exit non-zero, so these commands double as assertions in an automated reproduction run. Code-quality reproducibility is enforced separately through the developer command reference: `just lint` runs the Ruff linter over `src` and `scripts`, and the broader `just quality` recipe chains formatting, terminology, documentation, type, and security checks for a full pre-commit gate.

6.3 Manuscript Reproducibility

This manuscript is itself a reproducible artifact. Every quantitative value in the prose — the pipeline step count, the family and backend counts, the source and test inventories — is a token rather than a hard-coded literal, and the deterministic producer regenerates all of them from the tracked files at the current commit:

```
uv run python scripts/z_generate_manuscript_variables.py
```

That command recomputes the {{…}} tokens, persists them to `output/data/manuscript_variables.json` for audit, and hydrates the manuscript sources into `output/manuscript/`. The manuscript's own figures are rebuilt from the same token map:

```
uv run python -m scripts.manuscript_build_figures
```

The hydrated sources are then rendered to PDF by the docxology template's render stage. That stage lives in a separate checkout, with this repository symlinked into it at `projects/active/GeneralizedNotationNotation`; run from the template root:

```
uv run --frozen python scripts/pipeline/stage_03_render.py \
  --project GeneralizedNotationNotation
```

The render needs a LaTeX installation providing the packages listed in `manuscript/preamble.md` plus `seqsplit`; the template guards `seqsplit` with `\IfFileExists`, so a missing copy degrades rather than failing the build.

Because the variables file is regenerated before rendering, the counts in the rendered PDF track the repository state at the commit recorded in `output/data/manuscript_variables.json` (d20b76a54): a code change that alters, for example, the test inventory (651 test files, 6694 test functions) propagates into the prose on the next regeneration without any manual editing.

6.4 Reproducibility Contract

- Do not cite results that cannot be regenerated or directly traced to a command in this repository.
- Keep generated outputs under `output/` and maintained manuscript source under `manuscript/`. Regenerate owned build artifacts when their inputs change, and preserve the raw command receipts and numerical results needed to substantiate historical runs.
- Express every quantitative claim in the prose as a double-brace `{{...}}` token substituted by `scripts/z_generate_manuscript_variables.py`, never as a hard-coded number.
- Keep private data, credentials, and unpublished sensitive details out of the manuscript and out of version control.
- Record the exact verification commands — the smoke run, the acceptance and fidelity gates, and `just lint` — before marking this manuscript publication-ready.

7 Limitations and Next Steps

7.1 Current Limitations

GNN 4.1.0 is a working system with deliberately scoped boundaries, and it is worth stating those boundaries plainly rather than implying broader completeness than the artifacts support. The reliability gates that underpin our confidence in the system — the semantic-fidelity ledger and the cross-framework reliability ledger — are reproducible by command and recorded against the 9 model families, but this manuscript does not assert a particular full-suite pass rate as a headline number. Pass and skip counts shift with the local toolchain, the presence or absence of optional simulation dependencies, and whether the optional Ollama integration paths are exercised, so we retain the reproducing command together with its source, environment, selected-input, and artifact bindings, leaving run-specific tallies in the ledgers and execution traces. A fresh run supplies new evidence for its own environment; it does not replace a preserved historical receipt.

A second limitation lives at the boundary between the model families and the rendering backends. GNN registers 12 simulation backends, but coverage across the family-by-backend matrix is intentionally uneven, and the gaps are recorded as explicit *profiled-unsupported* statuses rather than silently omitted. The cross-framework acceptance check is anchored on the continuous family, where JAX, NumPyro, RxInfer.jl are compared directly; selected continuous and hierarchical lanes carry profiled-unsupported markers at the render and execute steps (Steps 11 and 12) for backends that cannot yet faithfully realize their continuous-state or temporal-depth structure. This is honest bookkeeping, not a defect to be hidden: an unsupported status with a recorded reason is more trustworthy than a forced translation that quietly misrepresents the model. It also reflects the state of the surrounding literature, where expected-free-energy formulations, variational-inference reductions, and hierarchical active-inference methods are still being actively refined rather than collapsed into one settled executable recipe [Champion et al., 2024, Nuijten et al., 2026a, Rangarajan and Rao, 2026]. The practical consequence is that not every model expressible in the GNN syntax can be executed on every registered backend today, and authors should consult the profiled-unsupported ledger that `scripts/run_model_family_acceptance.py` writes — the command is given in sec. 6 — before assuming a given family will round-trip through a given framework. The per-kind shape of that matrix matters as much as its gaps: the categorical-only backends report the continuous linear-Gaussian kind `unsupported` by declared capability rather than by accident, structural wrappers have no renderable model body and are reported `unsupported`, and the recursive corpus is reserved for bounded proposal-only `--autonomous` runs and holds no committed models yet — boundaries recorded where the kinds are defined (sec. 3.2) rather than discovered mid-run.

A third limitation is one of scope rather than capability. The formal account in sec. 3 fixes each kind's parameterization as given — the categorical tensors `A`, `B`, `C`, `D`, and `E` of the discrete kind and the system matrices, Gaussian prior, and optional control declarations of the linear-Gaussian kind — and states only belief-level inference: state and policy inference for the categorical kind (eq. 6, eq. 7) and fixed-parameter filtering and control for the continuous kind. GNN can also express *parameter* learning: a matrix may be declared as a latent variable with a Dirichlet prior, and the repository ships both a corpus model that does so (`input/gnn_files/learning/dirichlet_likelihood_learning.md`, which learns its likelihood matrix `A` from observations by joint variational message passing) and the renderer strategy that emits code for it (`src/gnn/render/rxinfer/_strategies_learning.py`). That corpus directory is registered in no manifest family — the unregistered set is exactly `input/gnn_files/learning`, `input/gnn_files/recursive`, `input/gnn_files/thrml` — so parameter learning is exercised by neither the semantic-fidelity nor the cross-framework ledger and carries no figure or table in this manuscript. We state it here as implemented-but-out-of-scope rather than leave the omission to be inferred from the family tables.

Experimental targets also have explicit admission boundaries. THRML performs fixed-action categorical posterior sampling with bounded independent or validated joint composition; it does not supply Gaussian inference or policy/EFE optimization. cpomdp accepts linear-Gaussian models with resource-admitted finite policy search, and rejects unsupported kinds or over-budget search rather than reducing the request silently. Both require explicit framework selection, so registration does not promote either into the default execution set.

Finally, several capabilities depend on optional dependencies that are not part of the minimal install. The simulation frameworks themselves, the audio sonification path, the LLM-enhanced analysis step, and the interactive visualization step all require extra packages that a lean checkout will not have, and readiness diagnoses distinguish absent packages, toolchains, unsupported Python, failed probes, and timeouts. Their recorded outcomes depend on the step's requirements; missing dependencies or required unfinished work do not establish successful execution. This keeps the core parse-validate-export-visualize spine runnable in constrained environments, but it means a full 0–24 traversal of all 25 steps reflects the optional surface that a particular machine has installed. We consider this an acceptable engineering trade for portability, but it is a limitation a reader should hold in mind when interpreting any single end-to-end run.

7.2 Next Steps

The roadmap is concrete and staged. The current software version is GNN 4.1.0. It builds on the three safe-by-design orchestration contracts introduced in GNN 3.0.0, which generate and validate data only, with no live infrastructure mutation. The first is *durable observation streams*: file- and array-backed stream manifests (content-checksummed) with replayable execution traces, so that an extended run can be observed, paused, and re-derived deterministically before any live sensor or device-backed stream is ever introduced. The second is *resumable run sessions*: run-session manifests with atomic

checkpoint/resume, status inspection, and cancellation-safe cleanup so that extended model-family acceptance runs can be interrupted and resumed without corrupting partial state. The third is *auditable container plans*: generating hardened container plans with an explicit static security-review and rollback descriptor, deliberately stopping short of mutating any real cluster. Each contract is covered by tests over real objects with negative controls and a strict acceptance gate, additive live wiring connects them to the session-acceptance and run-manifest paths, and three new Model Context Protocol tools expose them. The unifying discipline across all three is that orchestration becomes more capable only as fast as it becomes more inspectable.

The next major release pushes toward bounded autonomy, and it is gated behind the 3.0.0 orchestration work for good reason. The intent is to promote today's proposal-only candidate-scoring machinery — which already ranks candidate patches using the existing validators, model-family ledgers, and interpretability reports — toward *reviewed self-editing of GNN files*. Crucially, the design keeps a human in the loop: edits are proposed and applied only after explicit user approval, with no automatic source mutation, and the autonomy is wrapped in policy, rollback, and audit controls before any self-modifying or distributed action is permitted. This continues the same principle that governs the rest of the system, in the lineage of active-inference accounts of self-organizing systems that act to minimize free energy under explicit generative models [Friston, 2010, Parr et al., 2022]: an agent should expand its license to act only in proportion to the reviewability of what it does. The next-release items are not claimed as complete here; they are the recorded direction of travel, and each will be evidenced by its own gates when it lands, just as the 3.0.0 contracts are evidenced by theirs today. On the model-kind axis the same staging applies: extending native hierarchical rendering beyond the two-level strategies, broadening continuous reach across the remaining registered backends, and populating the reserved recursive corpus each arrive with their own gate evidence rather than by prose assertion.

8 Supplemental Source Surface

This supplement records the top-level source surfaces a reader or reviewer should inspect before turning any prose in the main manuscript into a verifiable claim. Each surface below is authored material under version control; the generated artifacts it produces are described separately so that the boundary between hand-written source and reproducible output stays explicit.

The `src/` tree is the executable core of GNN. It is organized into 44 Python packages spanning 759 source files and roughly 223521 lines of code. Alongside these packages sit the 25 top-level step modules (0–24) that implement the numbered pipeline as thin orchestrators delegating into the packages, each module owning one stage of the progression from a parsed GNN text model through visualization, type checking, code export, and executable cognitive simulation. A claim about declared pipeline behavior should identify its source implementation; a claim about actual execution additionally requires a source-bound command and result receipt.

The `input/` tree holds the model corpora that exercise the pipeline. Its `gnn_files/` subtree contains 12 corpus directories organized by model family, together totaling 38 example files — 29 discrete-state specifications and 9 continuous linear-Gaussian specifications, the partition the manuscript's model-kind accounting uses (sec. 3.2). The cold-start index `input/gnn_files/INDEX.md` catalogues every exemplar with its kind and task folder, and the `recursive/` directory is reserved for bounded `--autonomous` proposal-loop runs, holding no committed models. Every model file under `input/` lives in that subtree, so the 38-file count covers the whole tree. A `model_family_manifest.json` enumerates the 9 registered families and their target directories. This manifest is the authoritative registry that downstream steps and the manuscript variable producer read when they report family counts and cross-framework coverage; it should be consulted directly rather than inferred from directory listings.

The `scripts/` tree contains the thin orchestrators that gate and reproduce the project. These include acceptance scripts that evaluate scoped pipeline runs, reliability gates that check selected semantic and coverage contracts, and the manuscript variable producer (`src/gnn/manuscript/variables.py` driven from this layer) that emits the double-brace `{{...}}` token values consumed throughout the manuscript. Treating these scripts as the source of reproduction commands keeps reported numbers bound to what the code actually computes.

The `docs/` tree is the prose and reference surface, comprising 641 files of specification, tutorial, and design documentation for the GNN language and its Active Inference grounding [Smékal and Friedman, 2023]. It is the place to verify that a manuscript statement about GNN syntax or semantics matches the documented language rather than a convenient paraphrase.

The `output/` tree collects per-step pipeline artifacts: the data dumps, intermediate representations, validation reports, and the 7 figure artifacts committed under it. Generated build artifacts can be regenerated from bound inputs, while raw receipts and numerical results preserve evidence of particular runs. Reproduction requires their source and environment bindings; the directory location alone does not establish current-run evidence. Notably, `output/data/manuscript_variables.json` is where the manuscript's substituted token values are materialized.

8.1 Authored Source Versus Generated Output

`src/`, `scripts/`, `input/`, `docs/` and `manuscript/` are authored and reviewed. The `output/` tree holds generated artifacts and retained run evidence; `output/manuscript/` contains token-substituted copies of the authored `manuscript/` sections. Regenerate owned build artifacts when appropriate, while preserving the receipts required to support historical outcomes. The commands that regenerate the output surfaces are listed in sec. 6: the smoke run for the per-step artifacts, `scripts/z_generate_manuscript_variables.py` for the token map and the hydrated sections, `scripts.manuscript_build_figures` for the 7 manuscript figures, and the template render stage for the PDF. Which values become tokens is not a matter of taste: any quantity the producer can derive from a source surface is a token, `scripts/check_manuscript_tokens.py` fails the build when a section hard-codes one instead, and the resulting 93-entry map is committed at `output/data/manuscript_variables.json` for audit. External references are resolved from `manuscript/references.bib`, which the same gate cross-checks against every citation key in the prose. This manuscript quotes no private or unpublished material.

9 Symbols and Glossary

This glossary defines the Generalized Notation Notation (GNN) language constructs used to specify generative models, together with the Active Inference symbols those constructs carry across the model kinds of sec. 3.2. Definitions follow the GNN syntax specification and the exemplar specifications distributed with the repository — the discrete POMDP exemplars and the continuous linear-Gaussian exemplars alike — and they align with the standard formulations of both kinds [Smékal and Friedman, 2023, Da Costa et al., 2020, Parr et al., 2022].

9.1 Language Constructs

A GNN file is an ordered, UTF-8 Markdown document whose level-2 headers name required and optional sections. The strict schema validator enforces section order, declaration grammar, and connection syntax; the constructs below are the load-bearing sections a parser must recognize; they are catalogued in tbl. 6.

Table 6: The GNN language constructs a conforming parser must recognize, with the meaning each carries. The first rows are the required sections that pin a model's identity, variables, and factor graph; the optional sections that follow carry parameterization, ontology bindings, timing, equations, and provenance. The operator rows are the connection vocabulary the `Connections` section is written in — directed `A>B`, undirected `A-B`, and their labeled v1.1 forms — and `default=…` supplies initialization hints. Definitions follow the GNN syntax specification and the exemplar specifications under `input/gnn_files`, and the strict schema validator enforces section order and declaration grammar before any downstream step runs. The construct vocabulary is deliberately kind-agnostic: the same `StateSpaceBlock`, `Connections`, and operator syntax declare a discrete categorical model, a continuous linear-Gaussian model, or a composed multi-agent or hierarchical one — what differs between kinds is which matrix names the block declares, as tbl. 7 details.

Construct	Meaning
`## GNNSection`	Required short identifier for the model block (no spaces, e.g. `ActInfPOMDP`).
`## GNNVersionAndFlags`	Required version declaration (`GNN v1` or `GNN v1.1`) with optional flags.
`## ModelName`	Required human-readable title of the generative model.
`## StateSpaceBlock`	Required block declaring every variable and matrix as `NAME[dim, …, type=…]`, one per line — the block whose declared vocabulary fixes the model's kind.
`## Connections`	Required edge list relating state-space variables, expressing the model's factor graph.
`## ModelAnnotation`	Optional free-text description of the model's modalities, factors, and assumptions.
`## InitialParameterization`	Optional concrete numeric values for declared matrices and vectors.
`## ActInfOntologyAnnotation`	Optional bindings from each variable to a CamelCase Active Inference ontology term.
`## ModelParameters`	Optional key-value dimensions (e.g. `num_hidden_states`, `num_obs`, `num_actions`) consumed by code generators.
`## Time`	Optional dynamics declaration: a time variable plus `Dynamic/Static`, `Discrete/Continuous`, and `ModelTimeHorizon`.
`## Equations`	Optional LaTeX-rendered formulas defining model dynamics and the relationships between declared variables; round-tripped by the semantic-fidelity gate.
`## Footer`	Optional closing section that terminates the file and lets a reader or parser enter it from either end.
`## Signature`	Optional provenance block carrying a cryptographic signature over the specification.
`NAME[`d_1`,`d_2`,…,type=…]`	A variable or tensor declaration; dimensions are positive integers or named references, and a `type` (`float`, `int`, `bool`) is required.
`A>B`	Directed (causal) connection operator: edge from `A` to `B`, e.g. `D>s` (a prior conditions a hidden state) or `F>x` (a transition matrix drives a continuous state).
`A-B`	Undirected (bidirectional) connection operator, e.g. `s-A` (a hidden state participates in the likelihood mapping).

Construct	Meaning
`A>B:label` / `A-B:label`	A v1.1 annotated edge; the trailing label documents the relation and is preserved but may be ignored for structural validation.
`default=…`	A v1.1 declaration hint (`uniform`, `zeros`, `ones`, `eye`, `random`) supplying an initialization for a matrix or vector.

9.2 Symbols by Model Kind

The exemplar specifications declare the generative-model components of Active Inference, mapping each GNN variable to its probabilistic meaning [Da Costa et al., 2020, Smith et al., 2022, Parr et al., 2022]. Because the notation spans model kinds, the same letter can name different objects in different kinds — most prominently `F`, which is the variational free energy of eq. 6 in the discrete kind and the state-transition matrix of eq. 9 in the continuous kind — so each row of tbl. 7 names the kind its reading belongs to and the equation in sec. 3 that fixes it. These authored cross-references support review of agreement between the glossary and formal statement; their consistency still requires validation when either changes.

Table 7: The Active Inference symbols a GNN specification carries, scoped by model kind and bound to the equation of sec. 3 that fixes each role. Read the discrete-kind rows as the categorical generative-model tensors (likelihood, controlled transitions, log-preferences, prior over initial states, habit prior) with `s`, `o`, π, and `u` as the inference variables and `F` and `G` as the two free-energy functionals minimized at eq. 6 and eq. 7; read the continuous-kind rows as the linear-Gaussian parameterization — system matrices, covariances, Gaussian prior, and the optional closed-loop declarations of eq. 11. The rows where one letter carries two kind-scoped readings (`F`, `u`, `t`) are deliberate: the notation reuses a compact alphabet across kinds, and the declared matrix vocabulary of the `StateSpaceBlock` — not the letter alone — is what fixes the reading, which is exactly what the kind classification of sec. 3.2 computes. Use the table as a decoding key when reading a `StateSpaceBlock`: every declared matrix name should map to exactly one kind-scoped row here, and the `## ActInfOntologyAnnotation` bindings described below are what make that mapping machine-checkable downstream.

Symbol	Meaning
`A`	Discrete kind: likelihood (observation) matrix encoding $P(o \mid s)$, a column-stochastic map from hidden states to observation outcomes (eq. 2).
`B`	Discrete kind: transition tensor encoding $P(s' \mid s, u)$, one column-stochastic slice per action in the canonical `B[next_state][previous_state][action]` order (eq. 3).
`C`	Discrete kind: preference vector — log-preferences over observation outcomes that bias the agent toward preferred outcomes (eq. 4).
`D`	Discrete kind: prior vector over initial hidden states (eq. 5).
`E`	Discrete kind: habit vector — an initial policy prior over actions, entering the policy posterior (eq. 8).
`s`	Discrete kind: current hidden-state distribution; `s_prime` (`s'`) is the next hidden-state distribution.
`o`	Discrete kind: current observation, an integer index over outcome modalities.
π	Discrete kind: policy — a distribution over actions inferred from expected free energy (eq. 8).
`u`	Discrete kind: the selected (sampled) action, sampled from the policy posterior. Continuous kind: the control input added to the state each step (eq. 9).
`F`	Discrete kind: variational free energy, minimized during state inference (eq. 6) [Friston, 2010]. Continuous kind: the state-transition matrix of the linear-Gaussian model (eq. 9).
`G`	Discrete kind: expected free energy per policy, minimized during policy inference to score candidate actions (eq. 7) [Da Costa et al., 2020].
`t`	Discrete time step; the horizon T bounds the product in eq. 1. Continuous kind: the time index of the state-space model.
`H`	Continuous kind: observation matrix mapping the latent state to the observation (eq. 10).

Symbol	Meaning
`Q`	Continuous kind: process-noise covariance of the state evolution (eq. 9).
`R`	Continuous kind: observation-noise covariance of the readout (eq. 10).
`x`	Continuous kind: the Gaussian latent state x_t; `x[2,1,type=float]` in the navigation exemplar declares a two-dimensional position.
`y`	Continuous kind: the continuous observation read out linearly from the state (eq. 10).
`prior_mean`	Continuous kind: prior mean μ_0 over the initial latent state (eq. 9).
`prior_cov`	Continuous kind: prior covariance Σ_0 over the initial latent state (eq. 9).
`goal_mean`	Continuous kind: preferred state $\mu^\star$ the closed-loop controller steers toward (eq. 11).
`control_gain`	Continuous kind: scalar proportional gain k on the belief error in the closed loop (eq. 11).

9.3 Ontology Bindings and Implementations

The `## ActInfOntologyAnnotation` section binds each variable to a canonical term — `A=LikelihoodMatrix`, `B=TransitionMatrix`, `C=LogPreferenceVector`, `D=PriorOverHiddenStates`, `s=HiddenState`, `o=Observation`, π`=PolicyVector` in the discrete exemplars; `F=StateTransitionMatrix`, `H=ObservationMatrix`, `Q=ProcessNoiseCovariance`, `R=ObservationNoiseCovariance`, `prior_mean=PriorMean`, `prior_cov=PriorCovariance`, `goal_mean=PreferredState`, `control_gain=ControlGain`, `x=ContinuousHiddenState`, `y=ContinuousObservation`, `u=ControlInput` in the continuous exemplars — which downstream pipeline steps use for semantic analysis and validation. The bindings are per-kind by construction: a continuous exemplar binds `F` to `StateTransitionMatrix`, never to a free-energy term, so the ontology layer carries the same kind-scoped reading tbl. 7 fixes by hand. The same GNN specification feeds the project's rendering backends — including the 12 registered targets (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn), of which the 12 render targets with execution adapters (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn) can be selected at Step 12 subject to model-kind admission and runtime readiness — with per-kind reach recorded as explicit statuses (tbl. 4) — so that a model written once in this notation can be parsed, visualized, and executed across the 25-step pipeline (0–24) without restating its mathematics [Smékal and Friedman, 2023, Heins et al., 2022, de Felice et al., 2021].

10 References

Entries below are resolved by Pandoc from `manuscript/references.bib` at render time; `scripts/check_manuscript_tokens.py` fails the build on a citation key that file does not define.

Dmitry Bagaev and Bert de Vries. Reactive message passing for scalable bayesian inference, 2021. URL https://arxiv.org/abs/2112.13251.

Théophile Champion, Howard Bowman, Dimitrije Marković, and Marek Grześ. Reframing the expected free energy: Four formulations and a unification, 2024. URL https://arxiv.org/abs/2402.14460.

Lancelot Da Costa, Thomas Parr, Noor Sajid, Sebastijan Veselic, Victorita Neacsu, and Karl Friston. Active inference on discrete state-spaces: A synthesis. *Journal of Mathematical Psychology*, 99:102447, 2020. doi: 10.1016/j.jmp.2020.102447.

Giovanni de Felice, Alexis Toumi, and Bob Coecke. Discopy: Monoidal categories in python. *Electronic Proceedings in Theoretical Computer Science*, 333:183–197, 2021. doi: 10.4204/EPTCS.333.13.

Karl Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010. doi: 10.1038/nrn2787.

Conor Heins, Beren Millidge, Daphne Demekas, Brennan Klein, Karl Friston, Iain D. Couzin, and Alexander Tschantz. pymdp: A python library for active inference in discrete state spaces. *Journal of Open Source Software*, 7(73):4098, 2022. doi: 10.21105/joss.04098.

Magnus Koudahl, Thijs van de Laar, and Bert de Vries. Realising synthetic active inference agents, part i: Epistemic objectives and graphical specification language, 2023. URL https://arxiv.org/abs/2306.08014.

Wouter W. L. Nuijten, Mykola Lukashchuk, Thijs van de Laar, and Bert de Vries. What type of inference is active inference?, 2026a. URL https://arxiv.org/abs/2606.04935.

Wouter W. L. Nuijten, Thijs van de Laar, and Bert de Vries. Expected free energy-based planning as variational inference. *Transactions on Machine Learning Research*, 2026b. URL https://arxiv.org/abs/2606.20658.

Thomas Parr, Giovanni Pezzulo, and Karl J. Friston. *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior*. MIT Press, 2022. doi: 10.7551/mitpress/12441.001.0001.

Prashant Rangarajan and Rajesh P. N. Rao. Hierarchical active inference using successor representations, 2026. URL https://arxiv.org/abs/2604.15679.

Jakub Smékal and Daniel Ari Friedman. Generalized notation notation for active inference models. *Active Inference Journal*, 2023. doi: 10.5281/zenodo.22820299. URL https://zenodo.org/records/22820299. Initial publication describing the GNN framework.

Ryan Smith, Karl J. Friston, and Christopher J. Whyte. A step-by-step tutorial on active inference and its application to empirical data. *Journal of Mathematical Psychology*, 107:102632, 2022. doi: 10.1016/j.jmp.2021.102632.

Thijs van de Laar, Magnus Koudahl, and Bert de Vries. Realising synthetic active inference agents, part ii: Variational message updates, 2023. URL https://arxiv.org/abs/2306.02733.